

Using OpenStreetMap

for Urban Spatial Analysis



Press Space for next page →



Workshop agenda

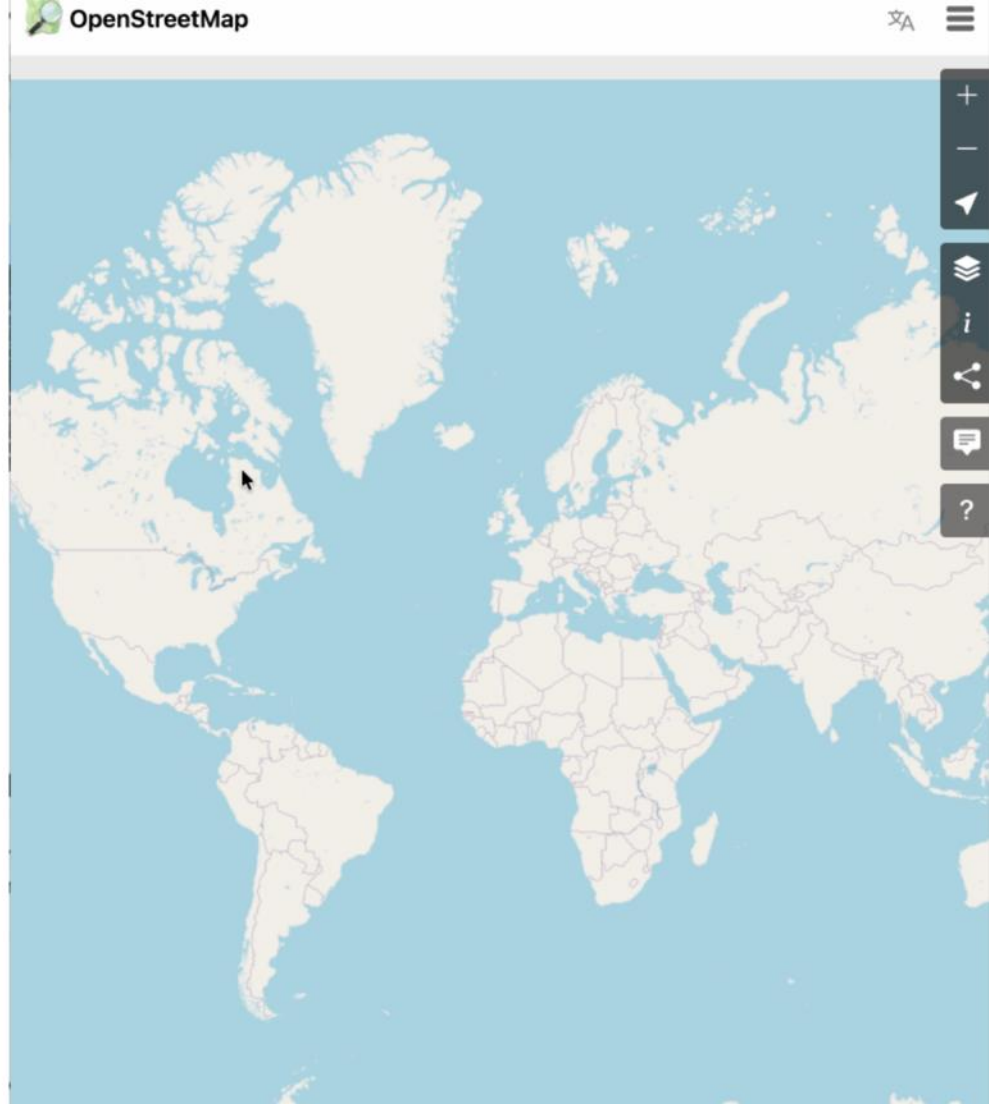
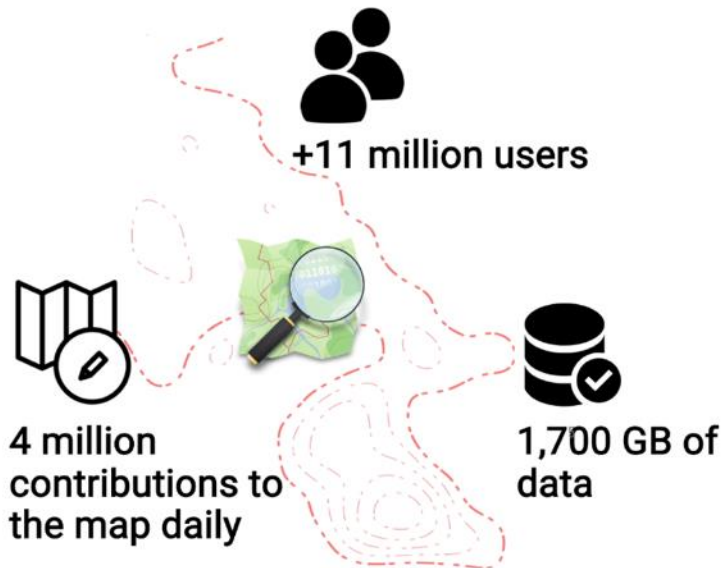
Time	Topic	Description
2 min	Welcome & Setup	Introductions, goals, software check
8 min	Introduction to OpenStreetMap	OSM data model, tagging
10 min	Exploring the OSM Ecosystem	Live demo of openstreetmap.org , contributions
15 min	Querying OSM Data	Hands-on: Overpass Turbo queries for study area
15 min	Alternative Visualization Methods	Overpass-ultra styling
5 min	Working with OSM Data in GIS	QuickOSM
5 min	Contributing to OSM	Hands-on: Create account, use iD editor

Learning objectives

-  Understand the OSM **data model**, including `nodes` , `ways` , `relations` , and the tagging schema
-  Write **OverpassQL queries** to extract targeted urban datasets (roads, buildings, amenities, land use)
-  Use **Overpass Turbo** and **Overpass-ultra** to download and script OSM data extraction workflows
-  Import OSM data into **QGIS** for spatial analysis and visualization
-  Create an OSM account and contribute edits using the **iD web editor**
-  Identify **research applications** of OSM data

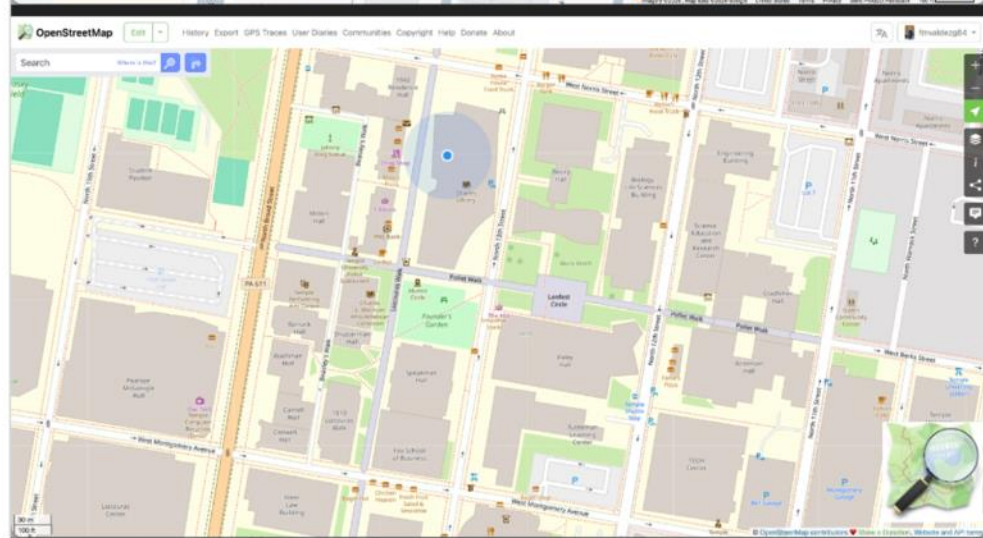
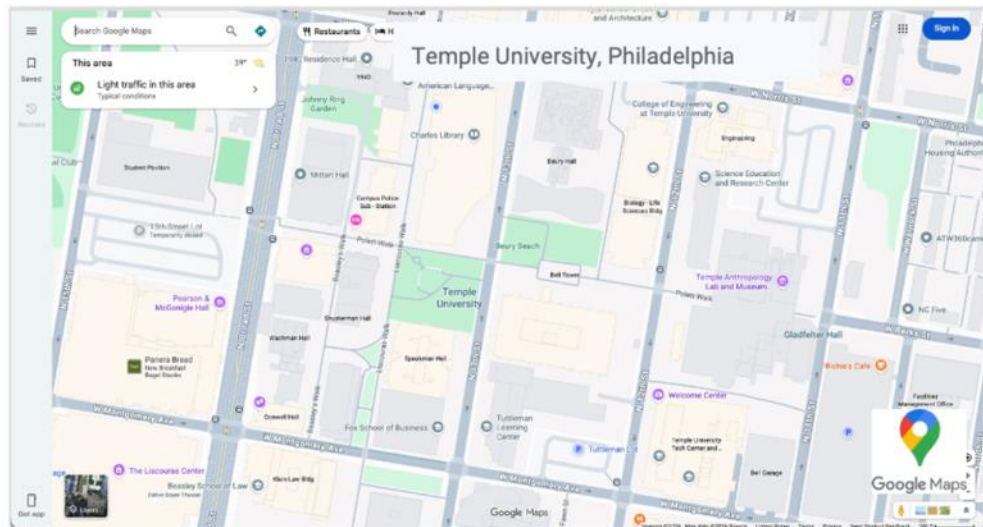
What is OpenStreetMap?

Is a free, open, and collaborative map of the world that is created and maintained by volunteers.



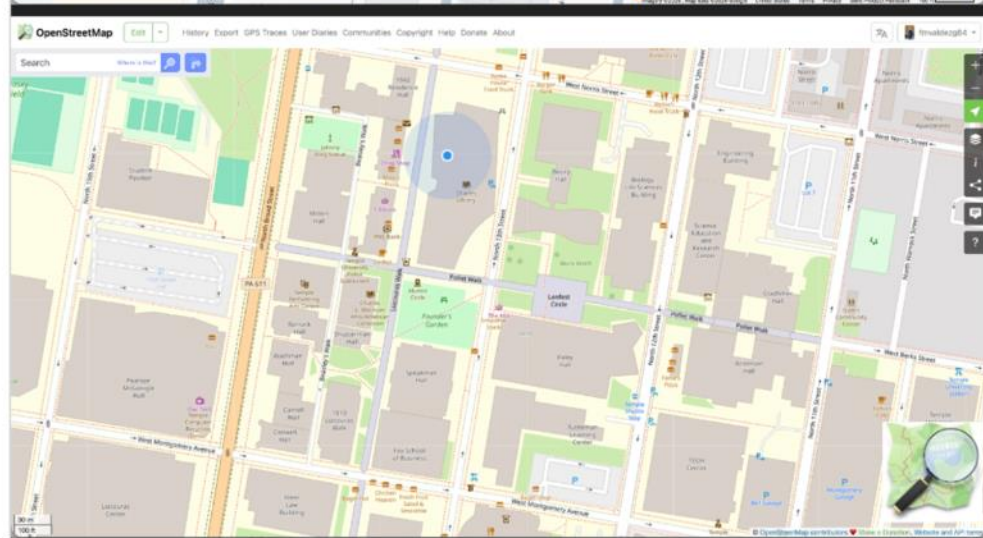
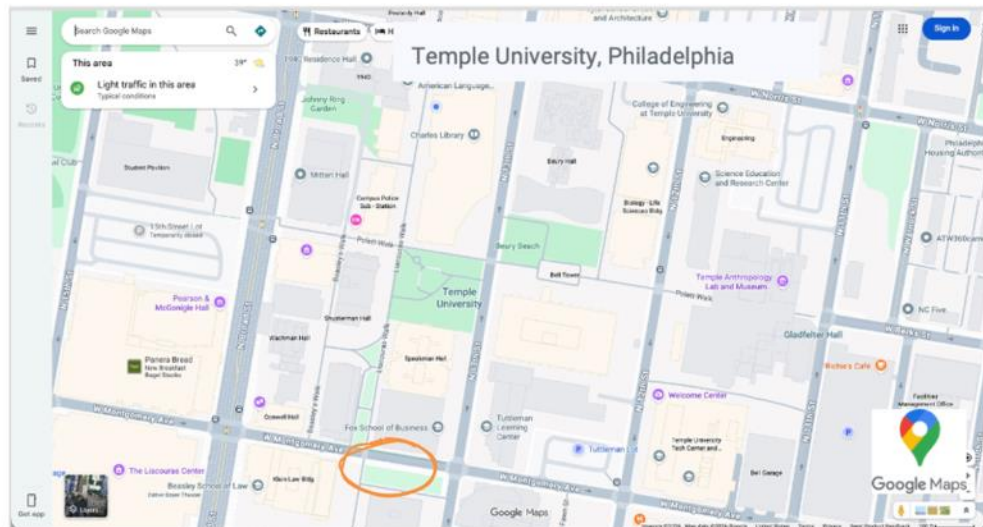
Why use OpenStreetMap?

- Free and Open
- Global Reach
- Local Knowledge
- Continuous Updates
- Agency
- Giving Back



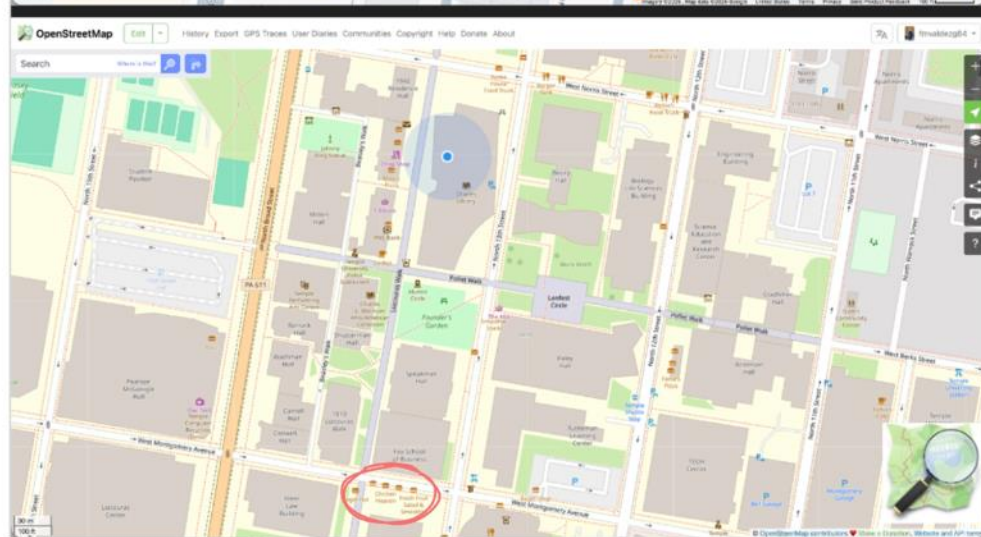
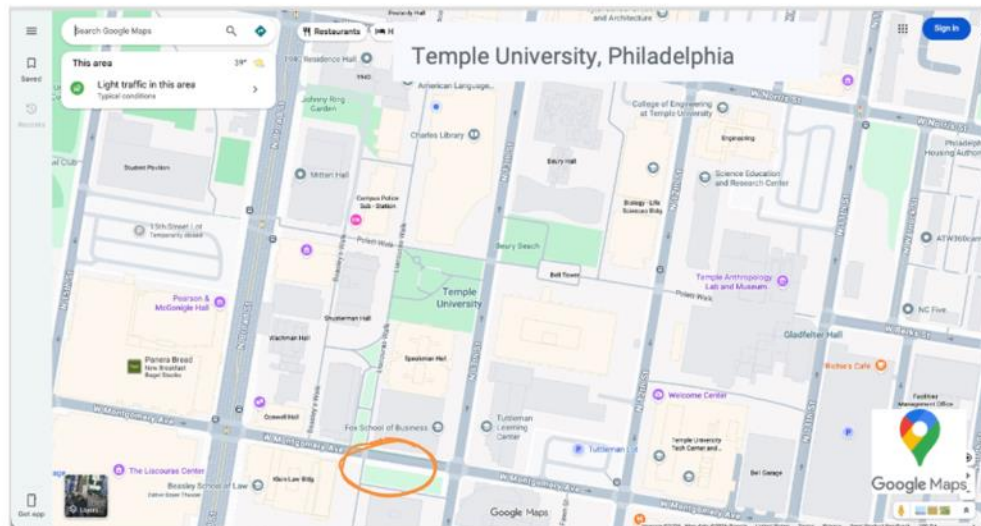
Why use OpenStreetMap?

- Free and Open
- Global Reach
- Local Knowledge
- Continuous Updates
- Agency
- Giving Back



Why use OpenStreetMap?

- Free and Open
- Global Reach
- Local Knowledge
- Continuous Updates
- Agency
- Giving Back



How OSM works?

How OSM works?

The Elements

How OSM works?

The Elements

- ◻ **Nodes** are dots used to mark locations.

How OSM works?

The Elements



Nodes are dots used to mark locations.



Ways are connected lines of nodes (roads, rivers.)

How OSM works?

The Elements

-  **Nodes** are dots used to mark locations.
-  **Ways** are connected lines of nodes (roads, rivers.)
-  **Closed ways** are ways that form a closed loop.

How OSM works?

The Elements

-  **Nodes** are dots used to mark locations.
-  **Ways** are connected lines of nodes (roads, rivers.)
-  **Closed ways** are ways that form a closed loop.
-  **Areas** are closed ways which are also filled.

How OSM works?

The Elements



Nodes are dots used to mark locations.



Ways are connected lines of nodes (roads, rivers.)



Closed ways are ways that form a closed loop.



Areas are closed ways which are also filled.



Relations create complex shapes or represent related but not physically connected elements.

How OSM works?

The Elements

-  **Nodes** are dots used to mark locations.
-  **Ways** are connected lines of nodes (roads, rivers.)
-  **Closed ways** are ways that form a closed loop.
-  **Areas** are closed ways which are also filled.
-  **Relations** create complex shapes or represent related but not physically connected elements.

Tags System

 **k=v**

key=value pairs describing elements

amenity=library

name=Charles Library

How OSM works?

The Elements

-  **Nodes** are dots used to mark locations.
-  **Ways** are connected lines of nodes (roads, rivers.)
-  **Closed ways** are ways that form a closed loop.
-  **Areas** are closed ways which are also filled.
-  **Relations** create complex shapes or represent related but not physically connected elements.

Tags System

 **key=value pairs describing elements**

```
amenity=library  
name=Charles Library
```

Examples:

- **Parking lot:** `amenity=parking`
- **Building:** `building=yes`
- **Residential road:** `highway=residential`
- **McDonald's:** `amenity=fast_food, brand=McDonald's, name=McDonald's`

Way: Charles Library ×

(708547491)

Version #8

Added tag details to Library

Edited 3 months ago by [fmvaldezg84](#)

Changeset [#174781286](#)

Tags

addr:city	Philadelphia
addr:housenumber	1900
addr:postcode	19122
addr:state	PA
addr:street	North 13th Street
amenity	library
building	yes
building:levels	4
description	This state-of-the-art learning center is transforming Temple's campus with its striking design and sophisticated technology. It's the most modern library in Pennsylvania and one of the most significant new libraries in North





Demo 1

Interacting with OpenStreetMap

Hands on:

OverpassQL queries

What is OverpassQL?

OverpassQL is a procedural, imperative query language that lets you search the OSM database and get back exactly the features you want.

The code is divided into **statements**, each ending with a semicolon `;`.

Statements execute sequentially, and each one changes the process's "execution state" — mainly by writing results into **sets**.

```
[out:json][timeout:25];  
node[amenity=drinking_water](39.88,-75.27,40.05,-75.02);  
out;
```


What is OverpassQL?

OverpassQL is a procedural, imperative query language that lets you search the OSM database and get back exactly the features you want.

The code is divided into **statements**, each ending with a semicolon `;`.

Statements execute sequentially, and each one changes the process's "execution state" — mainly by writing results into **sets**.

```
[out:json][timeout:25];  
node[amenity=drinking_water](39.88,-75.27,40.05,-75.02);  
out;
```

Line 1: Settings

`[name:value]`

`[out:json]`

Sets the output format

`[timeout:25]`

gives the server 25 seconds to respond

What is OverpassQL?

OverpassQL is a procedural, imperative query language that lets you search the OSM database and get back exactly the features you want.

The code is divided into **statements**, each ending with a semicolon `;`.

Statements execute sequentially, and each one changes the process's "execution state" — mainly by writing results into **sets**.

```
[out:json][timeout:25];  
node[amenity=drinking_water](39.88,-75.27,40.05,-75.02);  
out;
```

Line 2: Query

`node`

Requests point features

`[amenity=drinking_water]`

only return nodes with this ****tag****

`(39.95,-75.17,39.96,-75.16)`

bounding box (south, west, north, east)

What is OverpassQL?

OverpassQL is a procedural, imperative query language that lets you search the OSM database and get back exactly the features you want.

The code is divided into **statements**, each ending with a semicolon `;`.

Statements execute sequentially, and each one changes the process's "execution state" — mainly by writing results into **sets**.

```
[out:json][timeout:25];  
node[amenity=drinking_water](39.88,-75.27,40.05,-75.02);  
out;
```

Line 3: Output

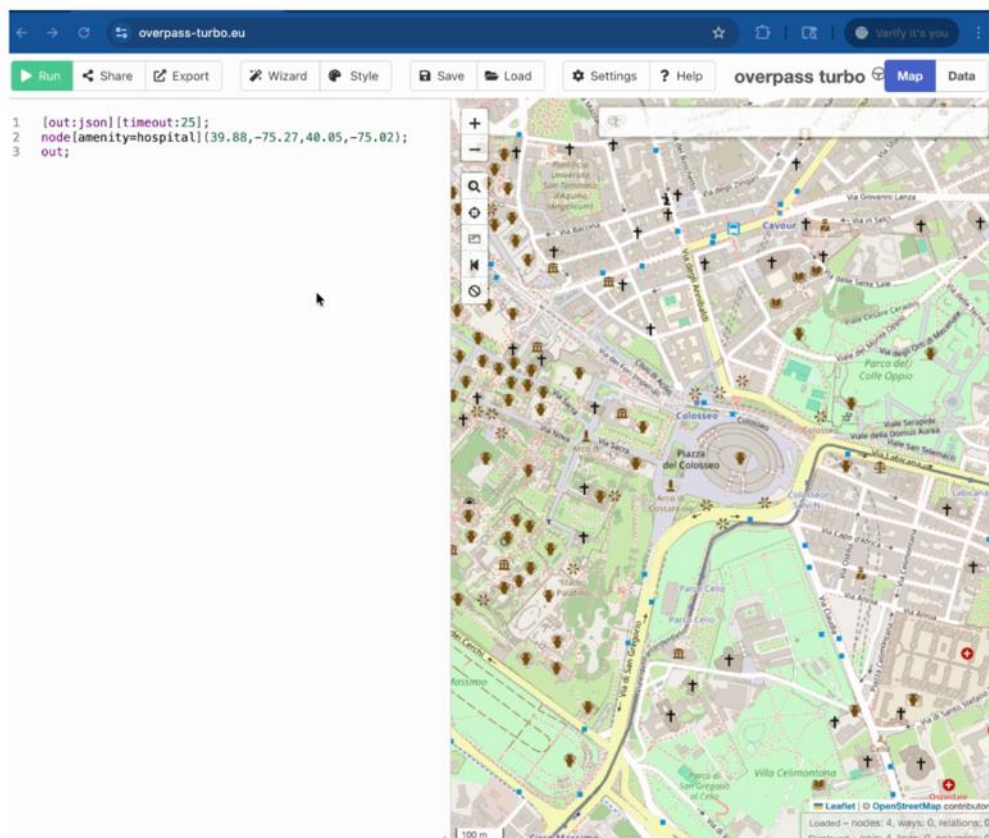
```
out;
```

Prints the results

It can be `ids` , `skel` , `body` , `tags` or `meta` .

Lets run the first query

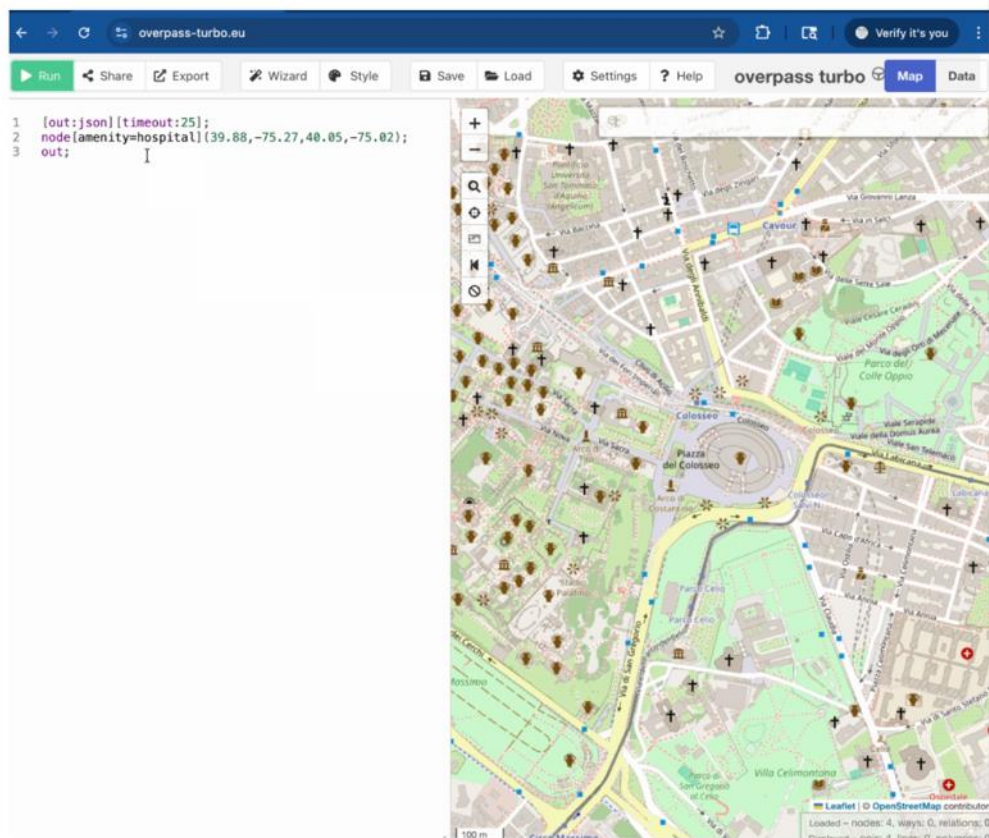
```
[out:json][timeout:25];  
node[amenity=hospital](39.88,-75.27,40.05,-75.02);  
out;
```



Lets run the first query

```
[out:json][timeout:25];
node[amenity=hospital](39.88,-75.27,40.05,-75.02);
out;
```

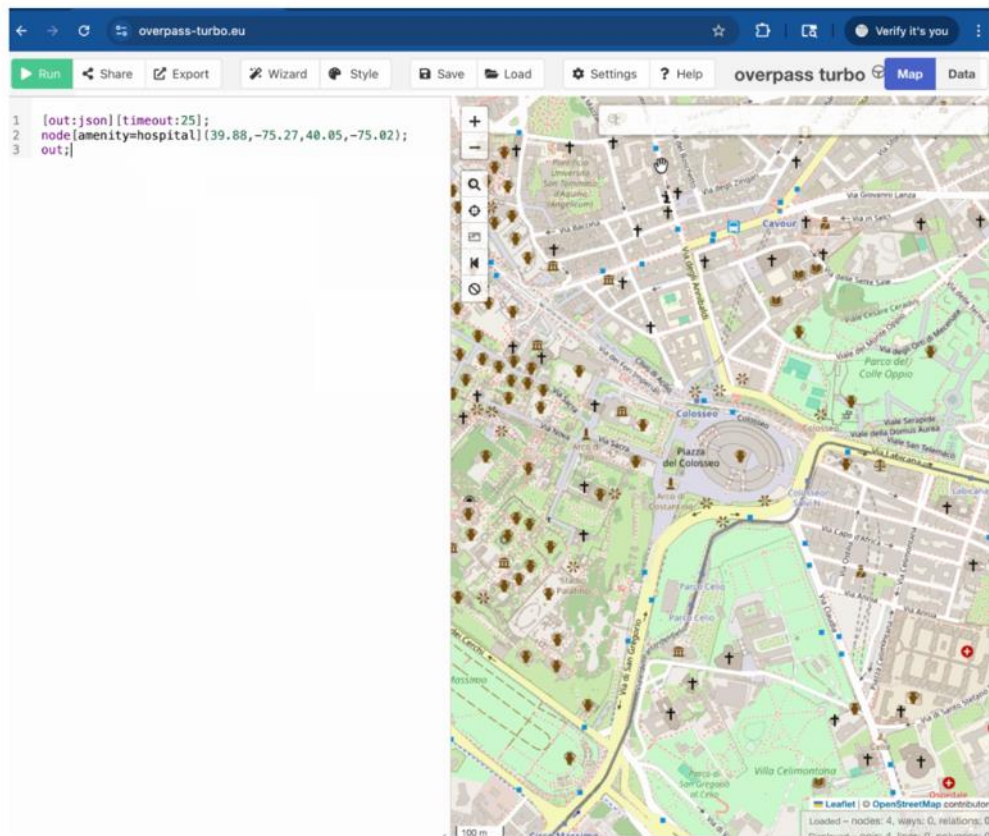
1. Copy the query



Lets run the first query

```
[out:json][timeout:25];  
node[amenity=hospital](39.88,-75.27,40.05,-75.02);  
out;
```

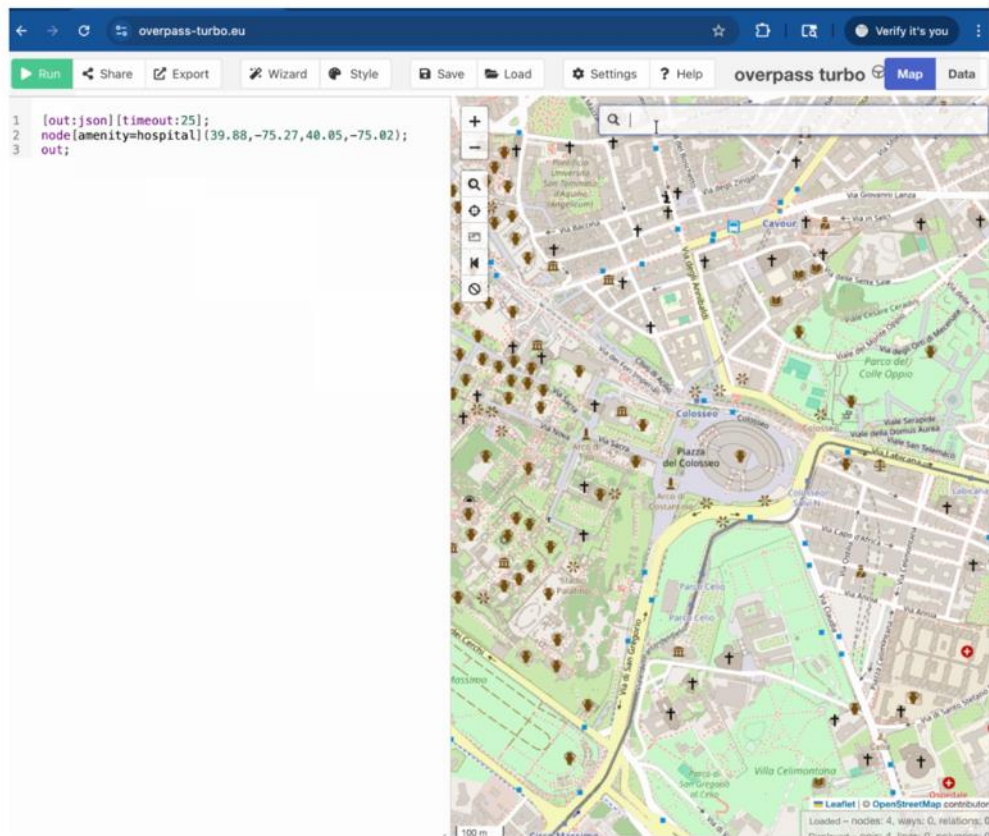
1. Copy the query
2. Got to <https://overpass-turbo.eu/>



Lets run the first query

```
[out:json][timeout:25];  
node[amenity=hospital](39.88,-75.27,40.05,-75.02);  
out;
```

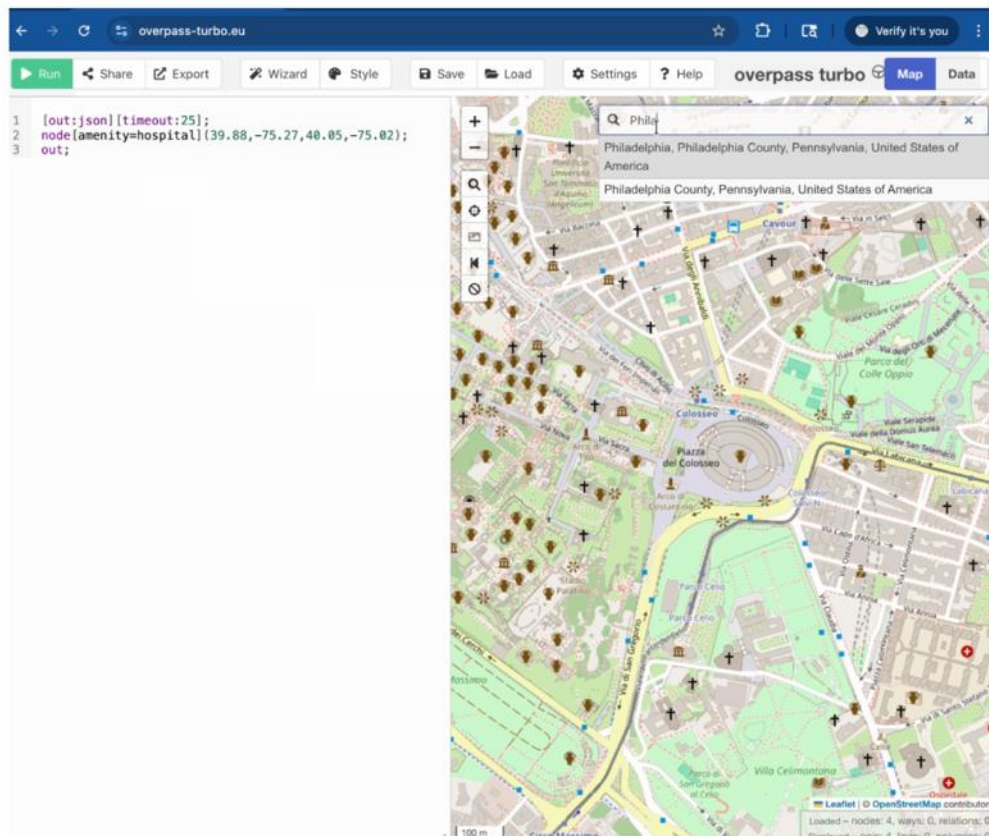
1. Copy the query
2. Got to <https://overpass-turbo.eu/>
3. Search and zoom in to Philadelphia, PA



Lets run the first query

```
[out:json][timeout:25];  
node[amenity=hospital](39.88,-75.27,40.05,-75.02);  
out;
```

1. Copy the query
2. Got to <https://overpass-turbo.eu/>
3. Search and zoom in to Philadelphia, PA
4. Paste the query on the text area on the left

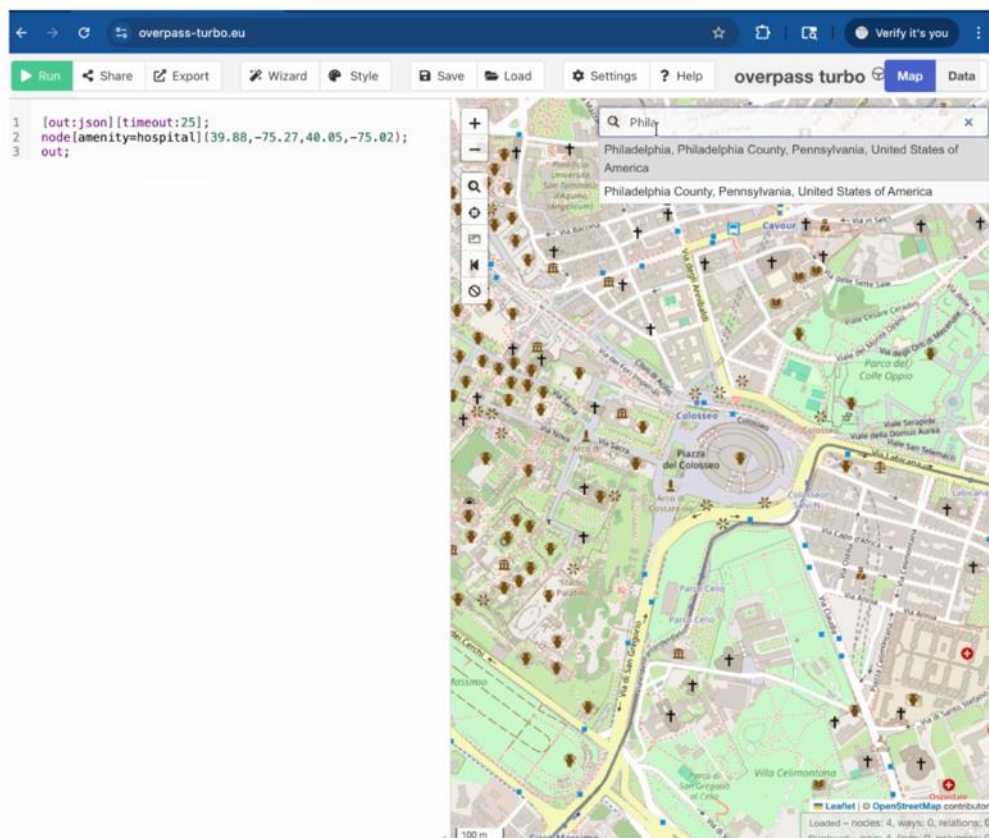


Lets run the first query

```
[out:json][timeout:25];  
node[amenity=hospital](39.88,-75.27,40.05,-75.02);  
out;
```

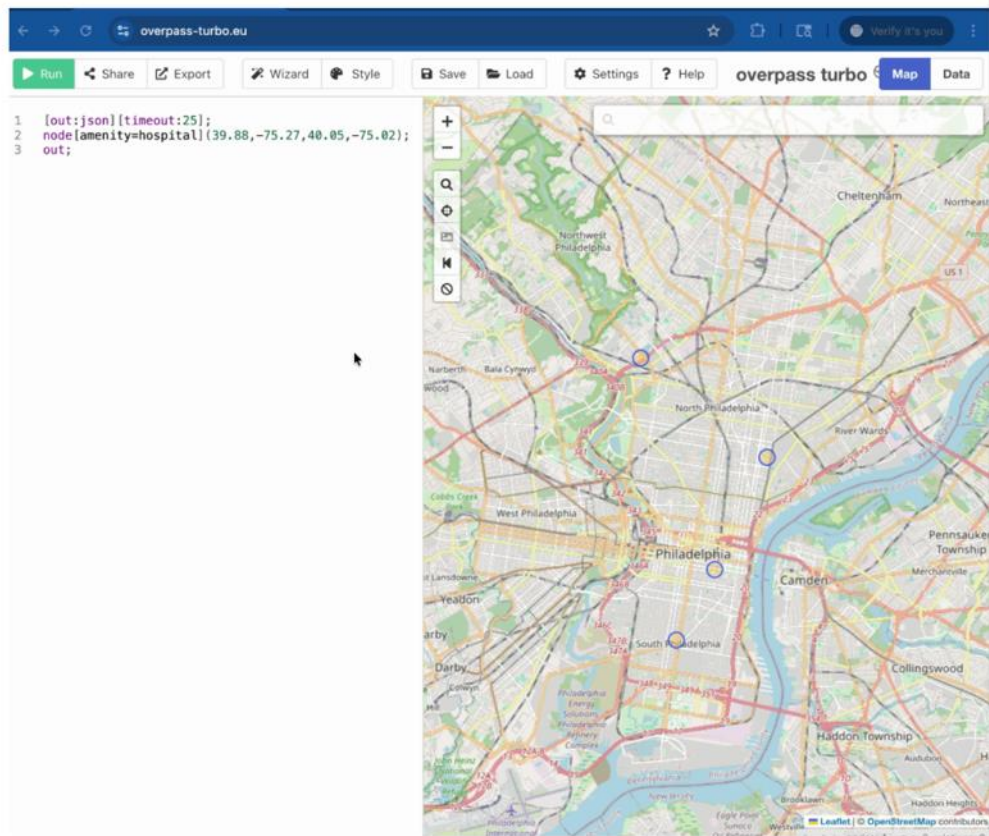
1. Copy the query
2. Got to <https://overpass-turbo.eu/>
3. Search and zoom in to Philadelphia, PA
4. Paste the query on the text area on the left
5. Click the run button

▶ Run



Wait!

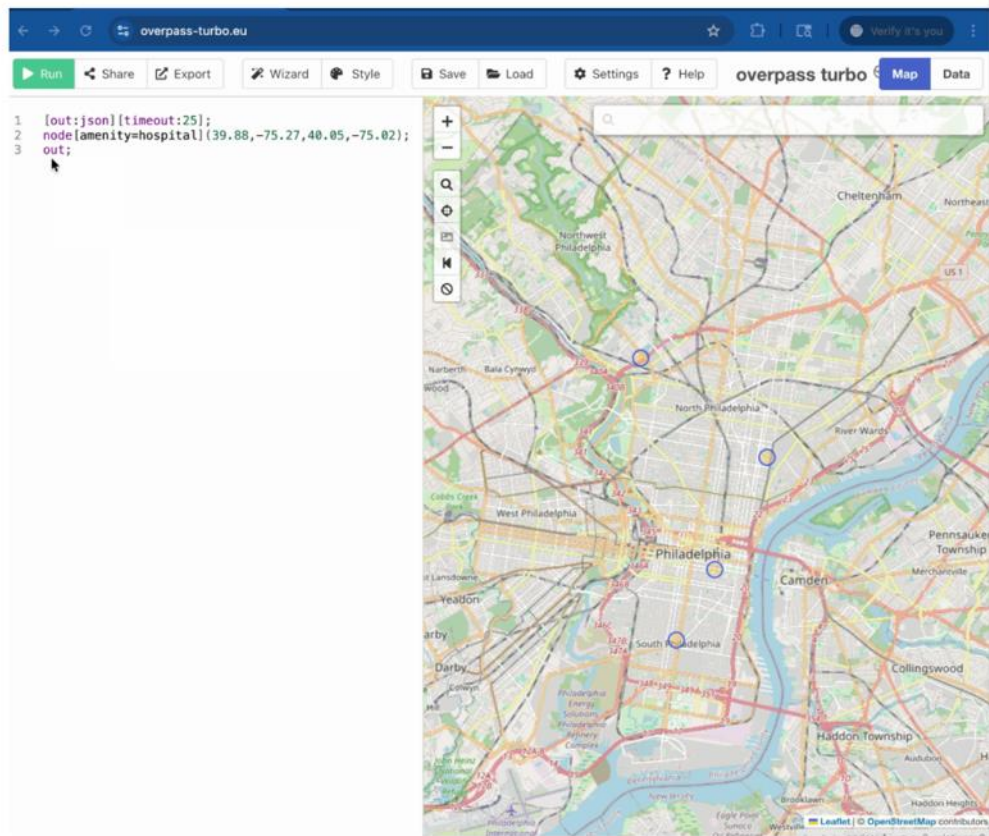
Where are all the hospitals in Philly?



Wait!

Where are all the hospitals in Philly?

```
[out:json][timeout:25];  
node[amenity=hospital](39.88,-75.27,40.05,-75.02);  
out;
```

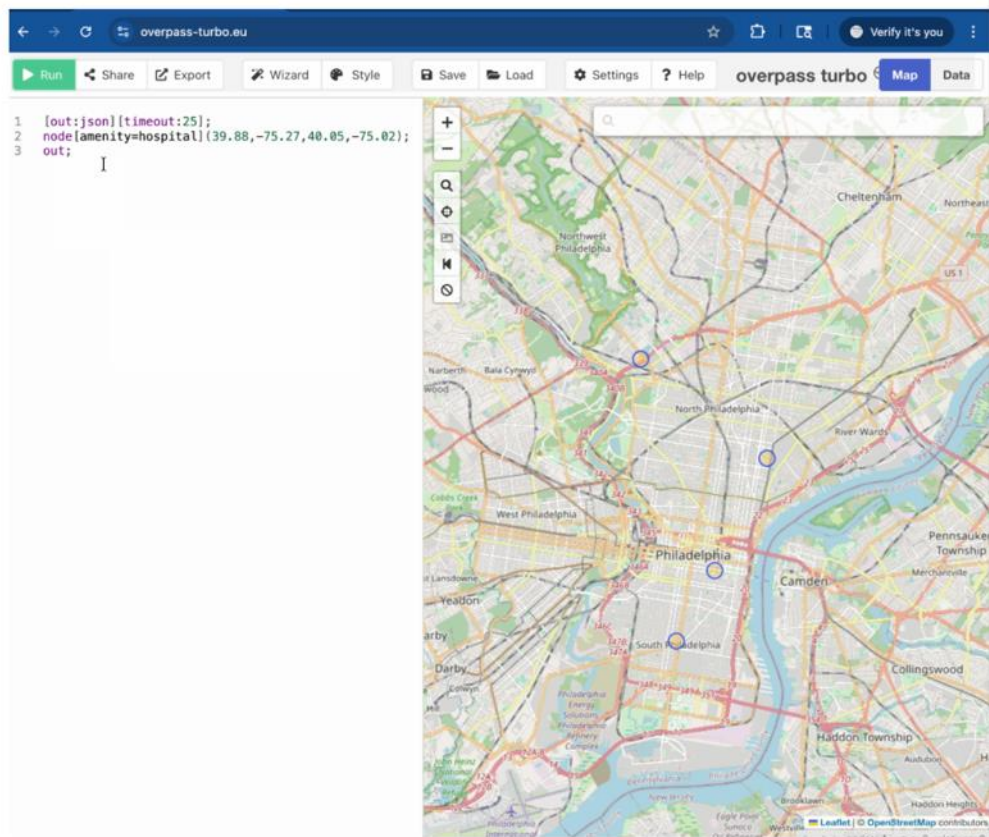


Wait!

Where are all the hospitals in Philly?

```
[out:json][timeout:25];  
node[amenity=hospital](39.88,-75.27,40.05,-75.02);  
out;
```

we only query **nodes** from the data set

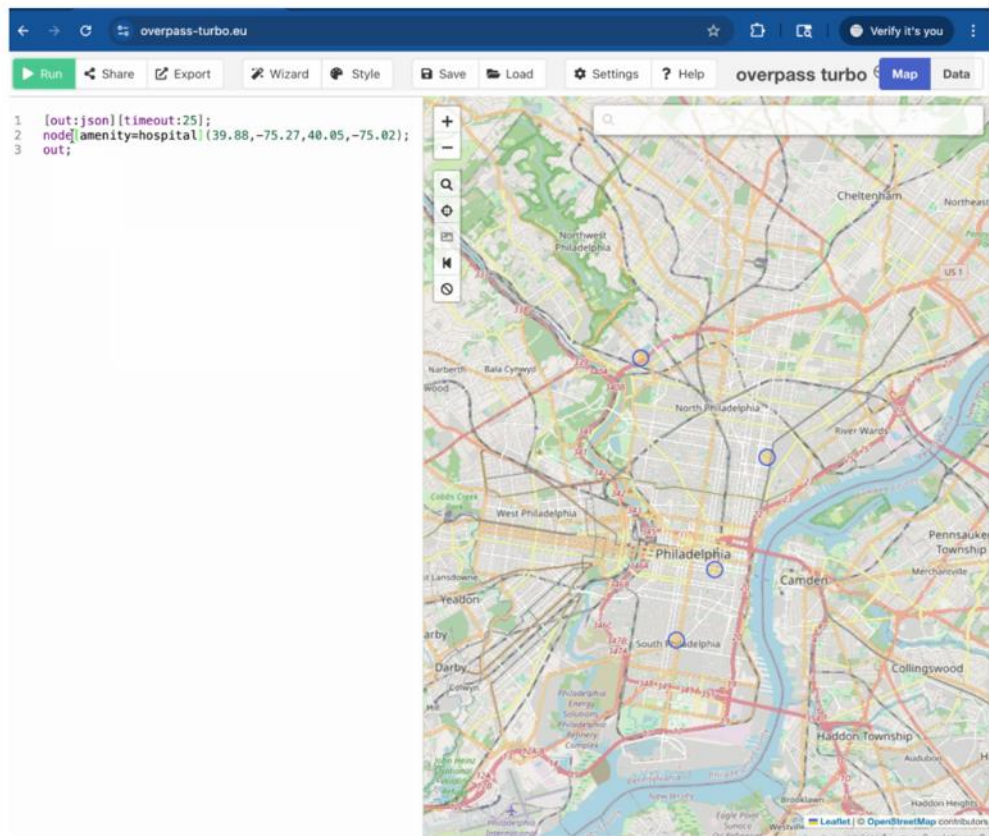


Wait!

Where are all the hospitals in Philly?

```
[out:json][timeout:25];  
nwr[amenity=hospital](39.88,-75.27,40.05,-75.02);  
out center;
```

we can replace `node` with `nwr` to query all types

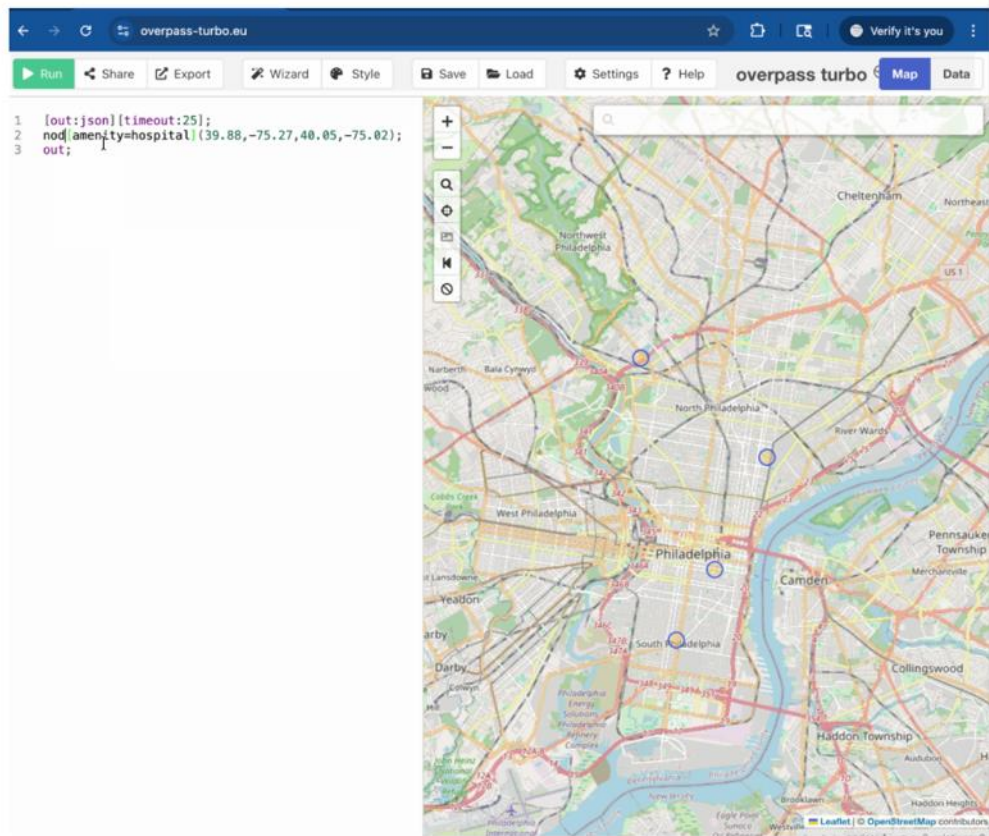


Wait!

Where are all the hospitals in Philly?

```
[out:json][timeout:25];  
nwr[amenity=hospital](39.88,-75.27,40.05,-75.02);  
out center;
```

we can replace `node` with `nwr` to query all types

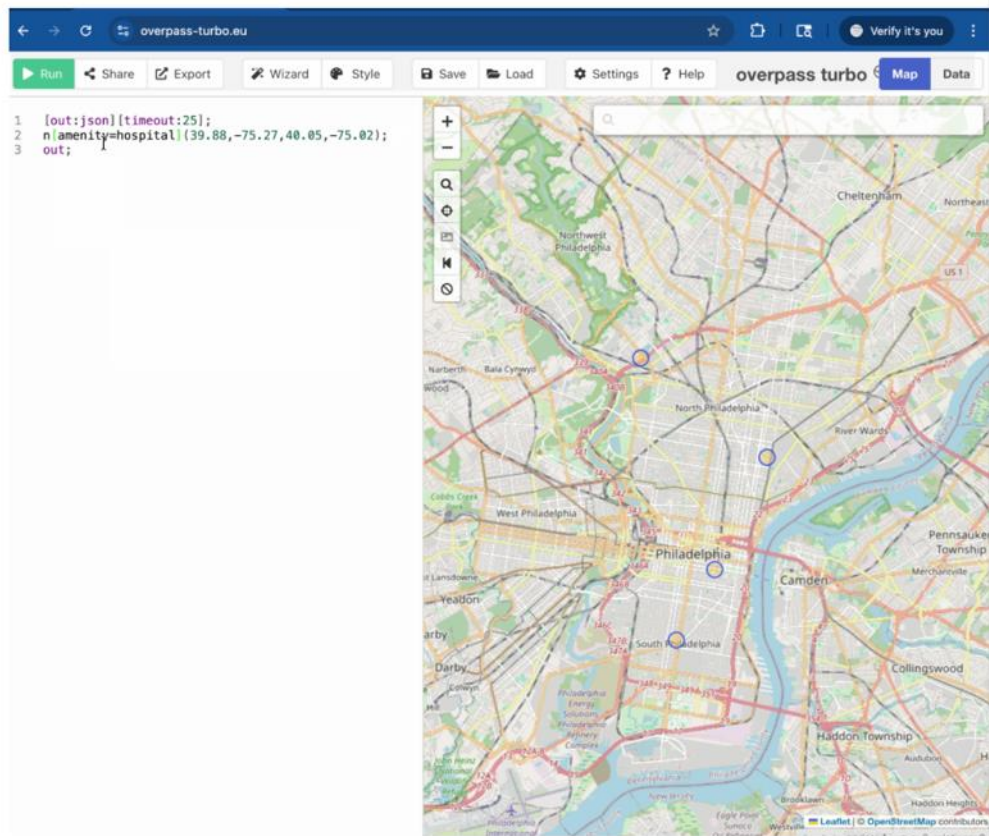


Wait!

Where are all the hospitals in Philly?

```
[out:json][timeout:25];  
nwr[amenity=hospital](39.88,-75.27,40.05,-75.02);  
out center;
```

we have to add `center` to the `out;` statement
to get a point on every element



Wait!

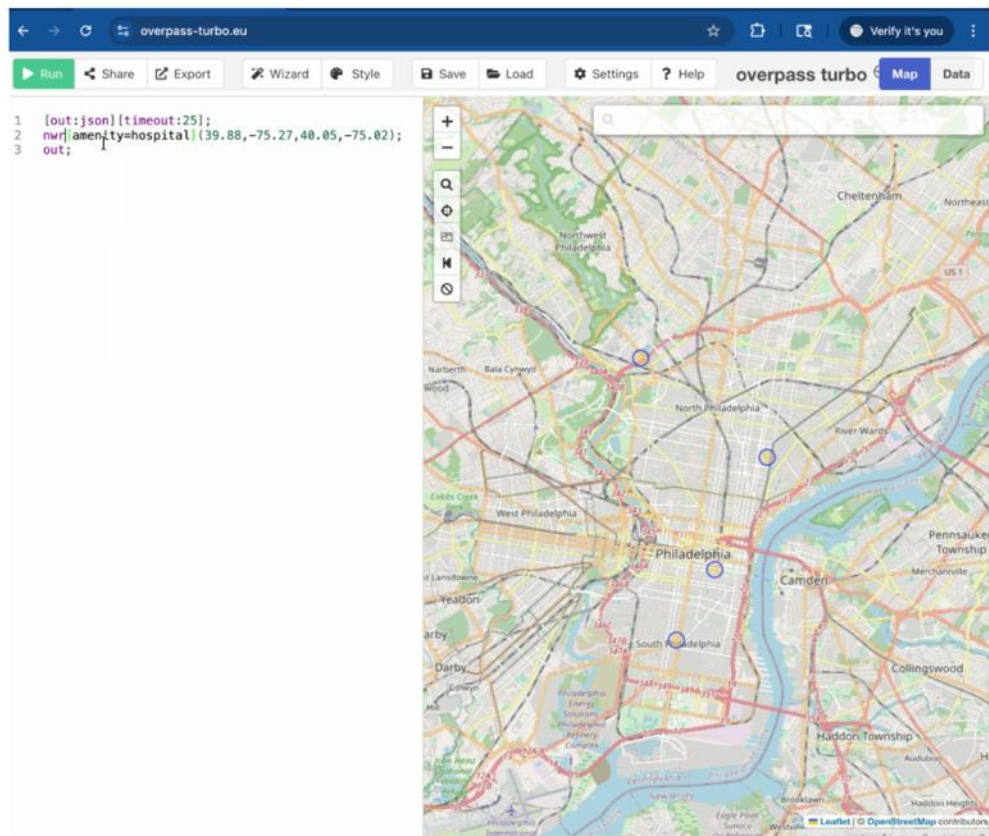
Where are all the hospitals in Philly?

```
[out:json][timeout:25];  
nwr[amenity=hospital](39.88,-75.27,40.05,-75.02);  
out center;
```

Re-run the query



Run



Run

Share

Export

Wizard

Style

Save

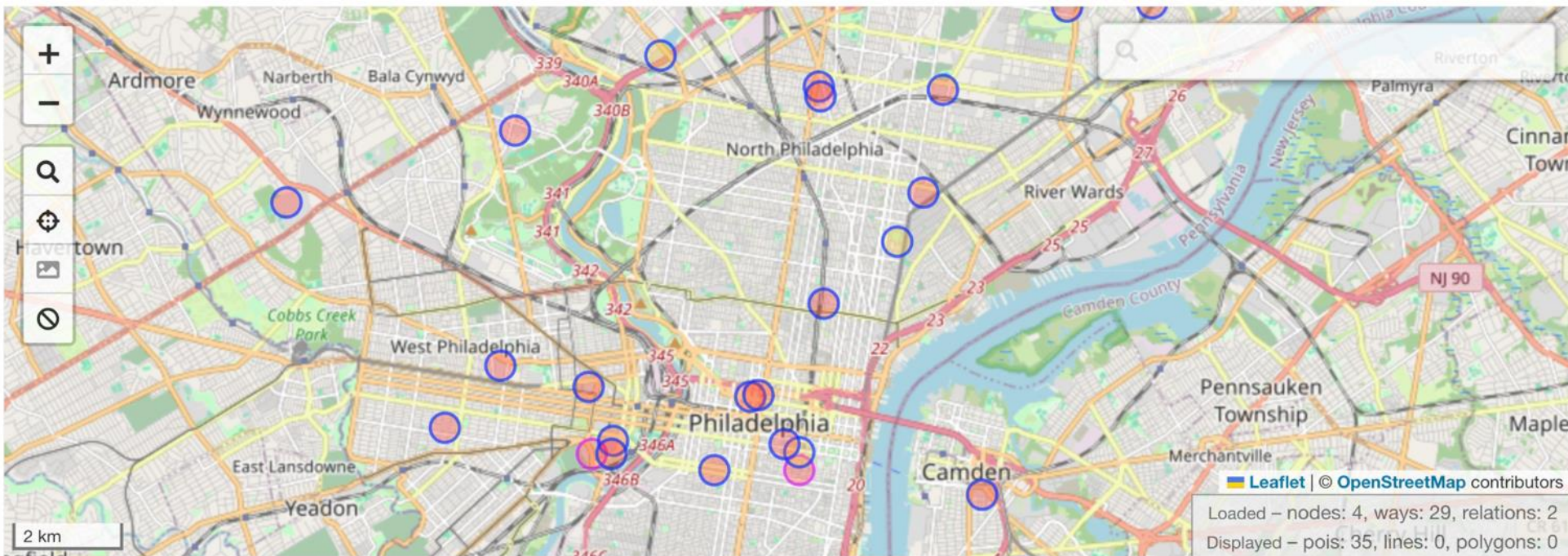
Load

Settings

Help

overpass turbo

```
1[out:json][timeout:25];  
2nwr[amenity=hospital](39.88,-75.27,40.05,-75.02);  
3out center;
```



Summing up

The anatomy of a query

```
[out:json][timeout:25];  
nwr[amenity=hospital](39.88,-75.27,40.05,-75.02);  
out center;
```


Summing up

The anatomy of a query

```
[out:json][timeout:25];  
nwr[amenity=hospital](39.88,-75.27,40.05,-75.02);  
out center;
```

Line 1: Settings

[out:json]

Output format

[timeout:25]

server response time

[Learn more about the settings here](#)

Summing up

The anatomy of a query

```
[out:json][timeout:25];  
nwr[amenity=hospital](39.88,-75.27,40.05,-75.02);  
out center;
```

Line 2: Query statement

node or way or nwr

type requested

[amenity=hospital]

tag requested

(39.95,-75.17,39.96,-75.16)

bounding box (south, west, north, east)

[Learn more about tags here](#)

Summing up

The anatomy of a query

```
[out:json][timeout:25];  
nwr[amenity=hospital](39.88,-75.27,40.05,-75.02);  
out center;
```

Line 3: Output

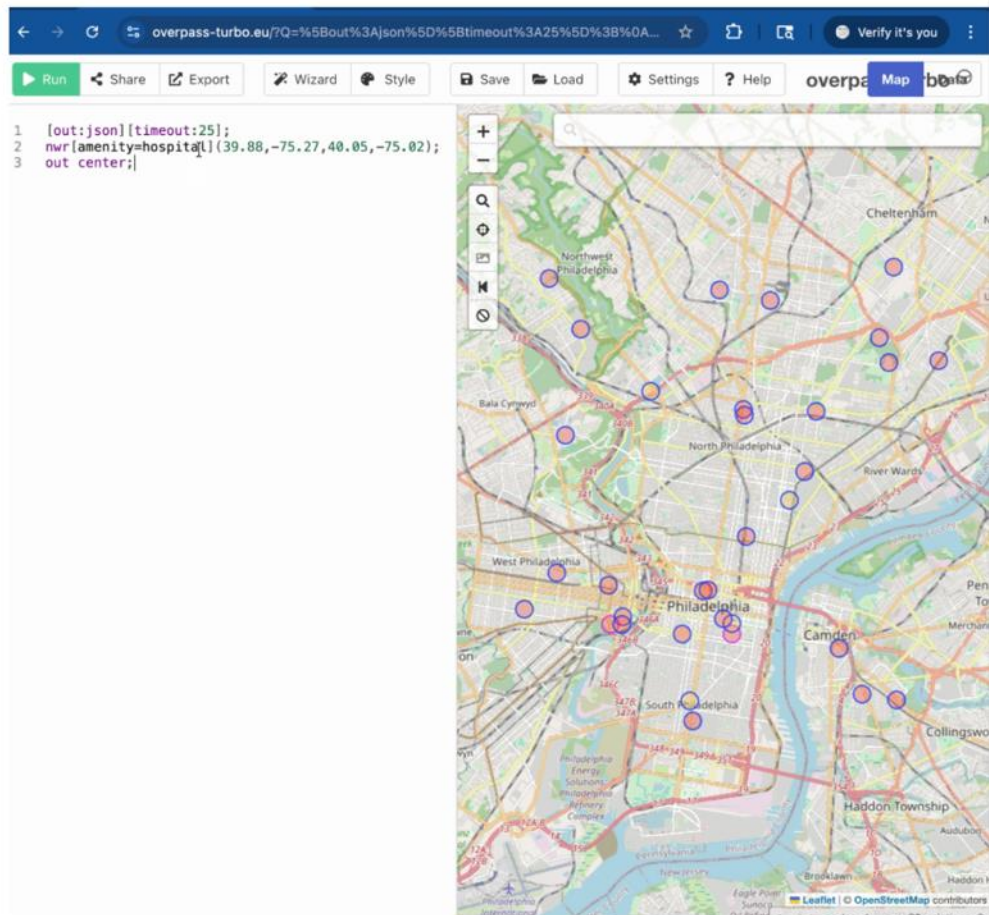
```
out center;
```

Prints the results

[Learn more about results here](#)

Try it out!

Create a new query using a different tag.

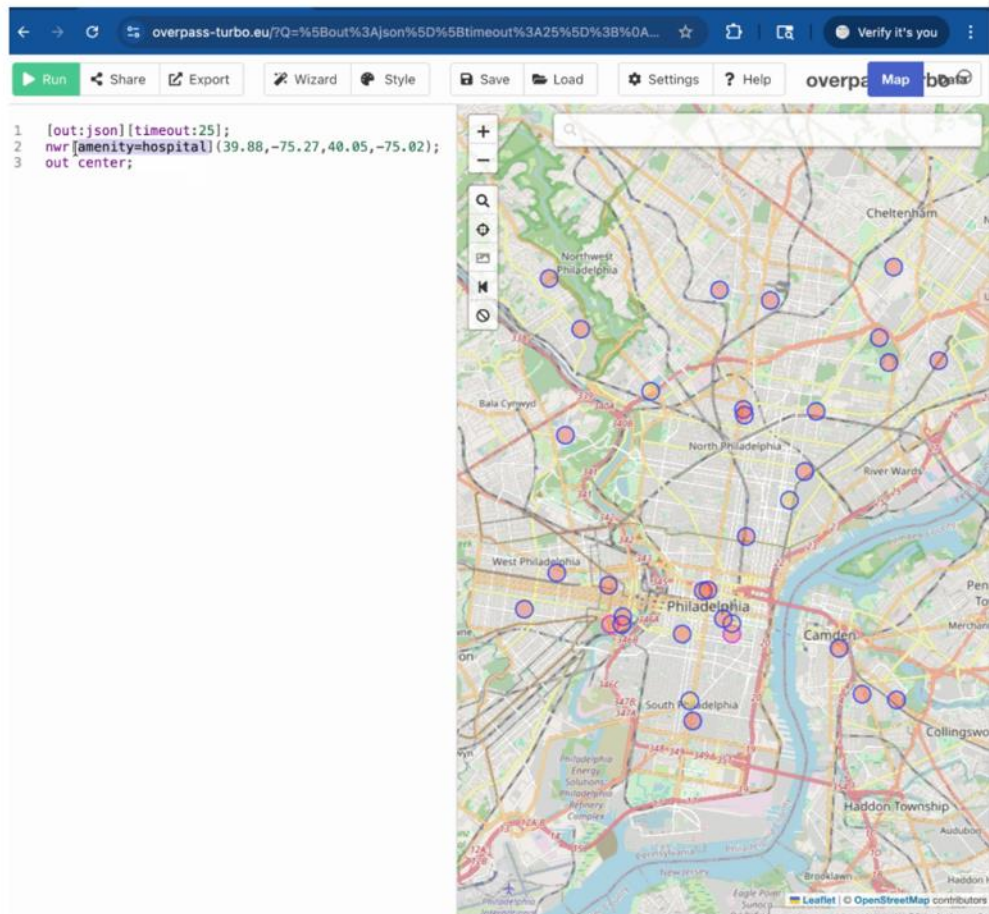


Try it out!

Create a new query using a different tag.

```
[out:json][timeout:25];  
nwr[leisure=park](39.88,-75.27,40.05,-75.02);  
out center;
```

all the parks

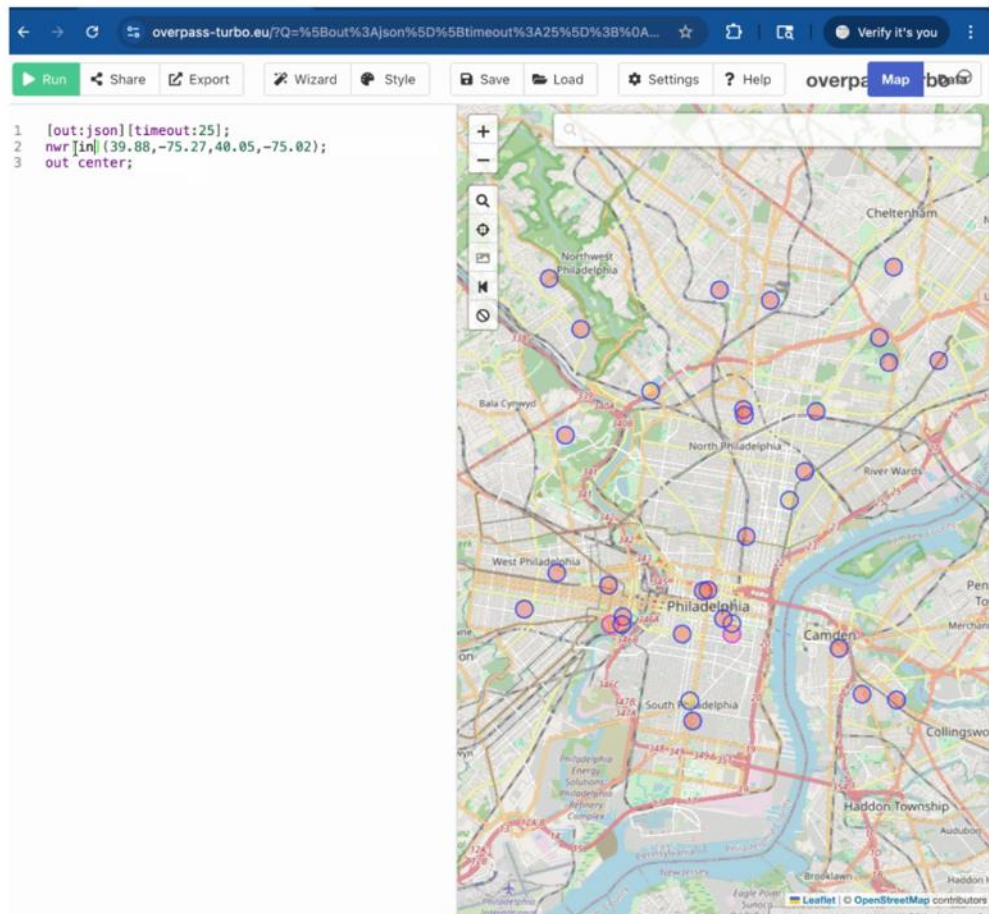


Try it out!

Create a new query using a different tag.

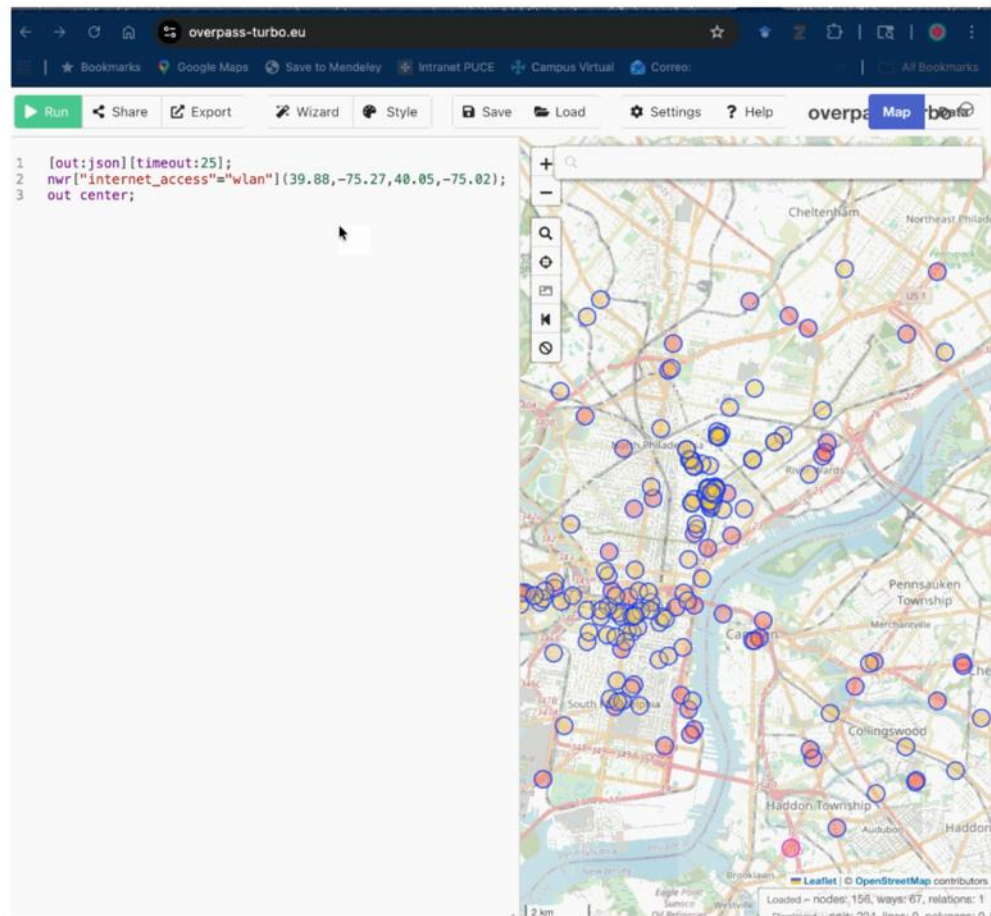
```
[out:json][timeout:25];  
nwr[railway=station](39.88,-75.27,40.05,-75.02);  
out center;
```

all railway stations



Combine tags

You can combine multiple tags in a single statement. They will work as an AND connector.



The screenshot shows the Overpass Turbo web application interface. The browser address bar displays `overpass-turbo.eu`. The top navigation bar includes buttons for `Run`, `Share`, `Export`, `Wizard`, `Style`, `Save`, `Load`, `Settings`, and `Help`. The left sidebar contains a code editor with the following query:

```
1 [out:json][timeout:25];  
2 nwr["internet_access"="wlan"] (39.88,-75.27,40.05,-75.02);  
3 out center;
```

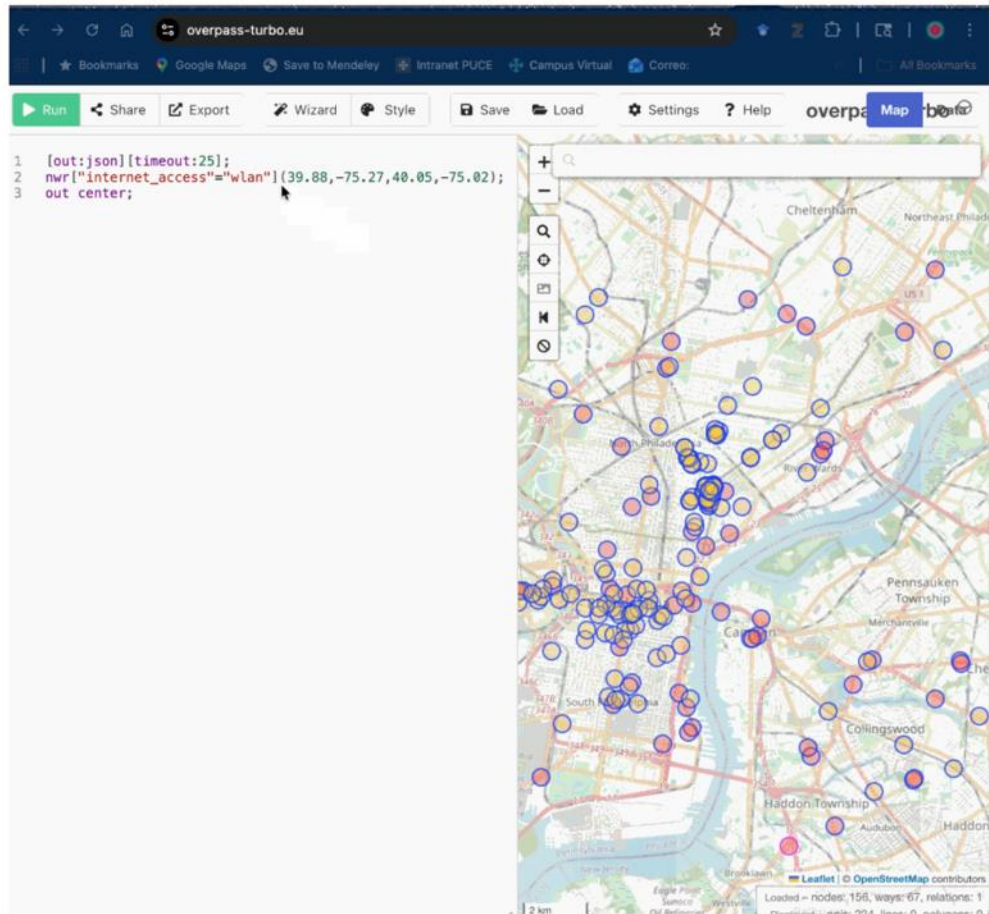
The main map area displays a map of Philadelphia with numerous circular markers. The markers are colored blue and red, representing different data points. The map includes labels for various locations such as Cheltenham, Northeast Philadelphia, Philadelphia, Rowley, Pennsauken Township, Collingswood, Haddon Township, and Audubon. The bottom status bar indicates the map is loaded with 156 nodes, 67 ways, and 1 relation.

Combine tags

You can combine multiple tags in a single statement. They will work as an AND connector.

```
[out:json][timeout:25];  
nwr["internet_access"="wlan"]["internet_access:fee":  
out center;
```

all places that have internet access wlan
AND are free to use

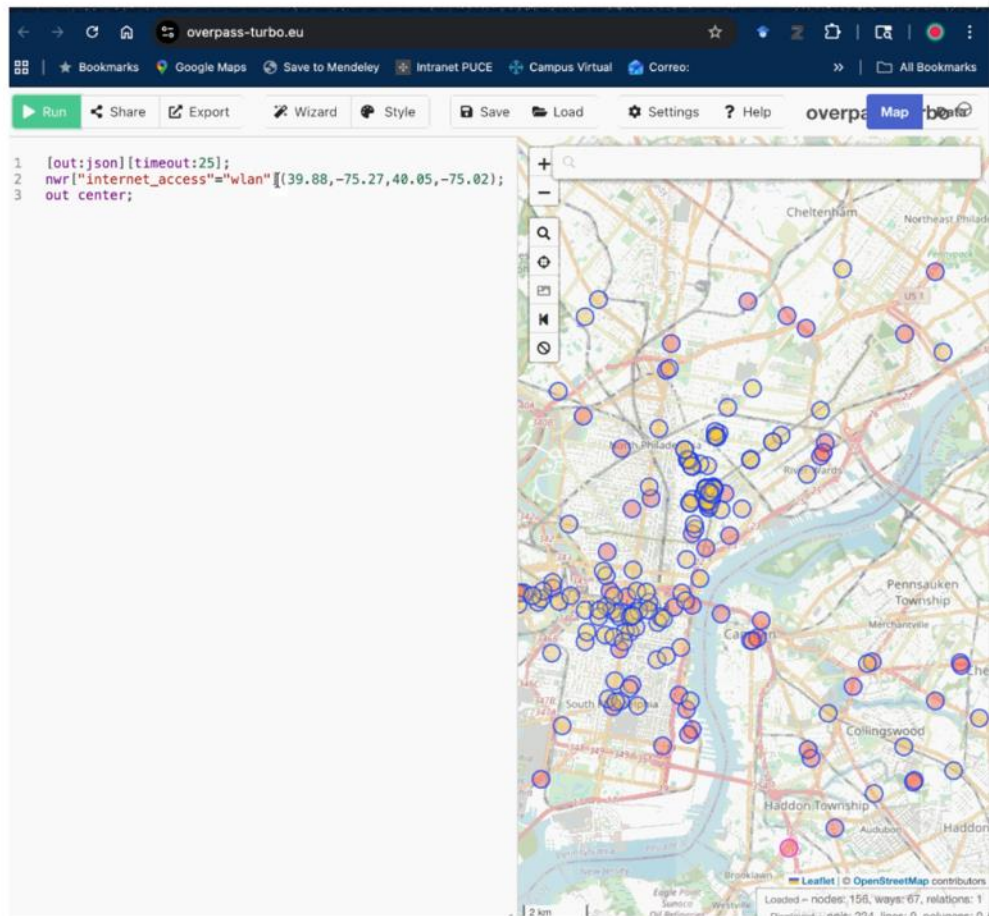


Combine tags

You can combine multiple tags in a single statement. They will work as an AND connector.

```
[out:json][timeout:25];  
nwr["leisure"="park"]["dog"](39.88,-75.27,40.05,-75  
out center;
```

all the parks
that include a `dog` tag

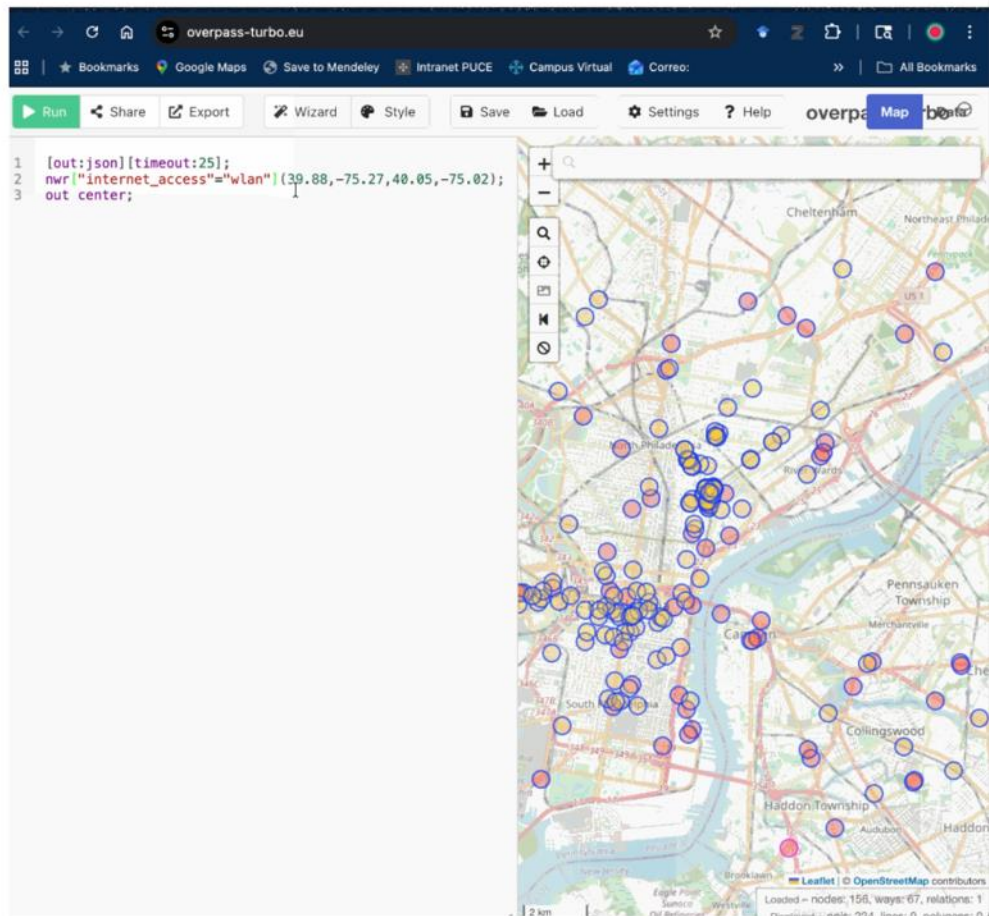


Combine tags

You can combine multiple tags in a single statement. They will work as an AND connector.

```
[out:json][timeout:25];  
nwr["railway"="station"]["subway"="yes"](39.88,-75.27,40.05,-75.02);  
out center;
```

all railway stations
from the subway system



Tag Filtering with Brackets

Tags are filtered using brackets. Here are the main patterns:

Basic Patterns

```
node[amenity=restaurant]; // Exact match  
node[name]; // Key exists  
node[!name]; // Key does NOT exist  
node[amenity!=restaurant]; // Not equal
```

Regular Expressions

```
node[name~"^Starbucks"]; // Regex match  
node[name~"starbucks",i]; // Case insensitive  
node[~"addr:.*"~".*Philly.*"]; // Key AND value
```

Multiple Conditions (AND)

```
node[amenity=restaurant][cuisine=mexican];  
Finds restaurants AND Mexican cuisine
```

Spatial Filters (Where to Search)

1 Bounding Box

Four coordinates directly after the type:

```
node[amenity=cafe](39.94,-75.18,39.96,-75.16);
```

2 Named Area

Search by area name:

```
area[name="Philadelphia"]->.searchArea;  
node[amenity=cafe](area.searchArea);
```

Line 1 finds Philadelphia → stores in searchArea. Line 2 queries cafes within that area.

3 Proximity (Around)

Find features within radius (meters) of others:

```
node[amenity=hospital](area.searchArea);  
node[amenity=pharmacy](around:500);
```

Finds pharmacies within 500m of hospitals

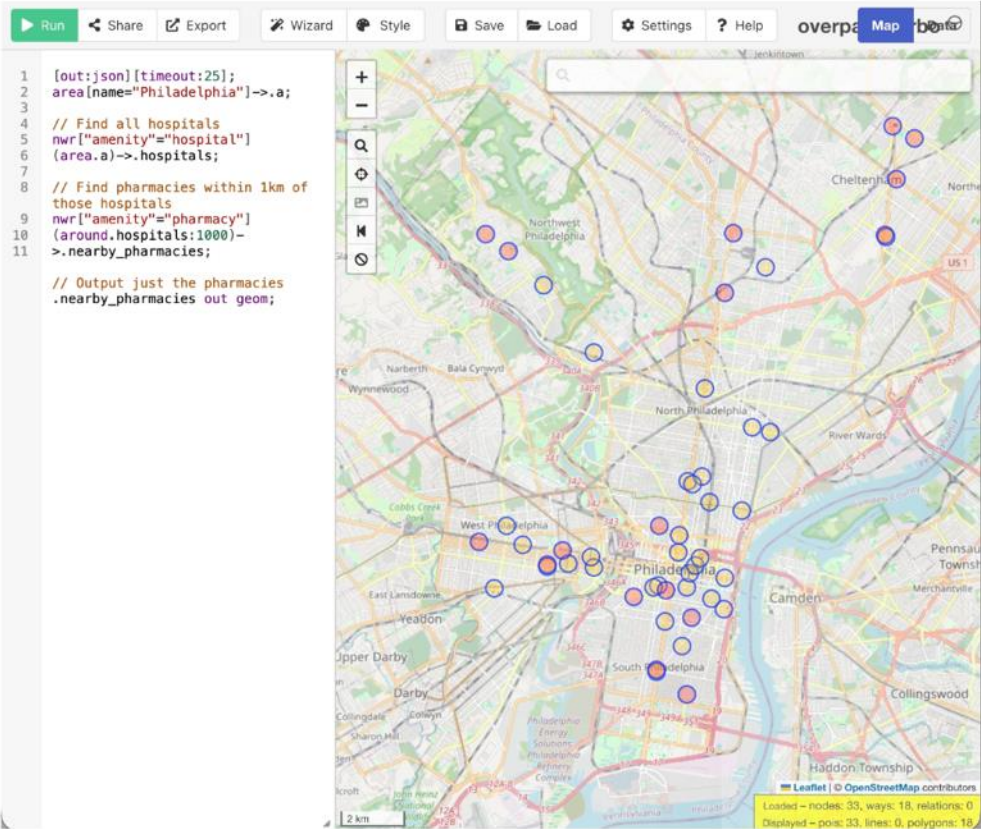
Lets try something out

Find pharmacies within 1km from hospitals

```
[out:json][timeout:25];
area[name="Philadelphia"]->.a;

// Find all hospitals
nwr["amenity"="hospital"](area.a)->.hospitals;

// Find pharmacies within 1km of those hospitals
nwr["amenity"="pharmacy"](around.hospitals:1000)->.nearby_pharmacies out center;
```



Lets try something out

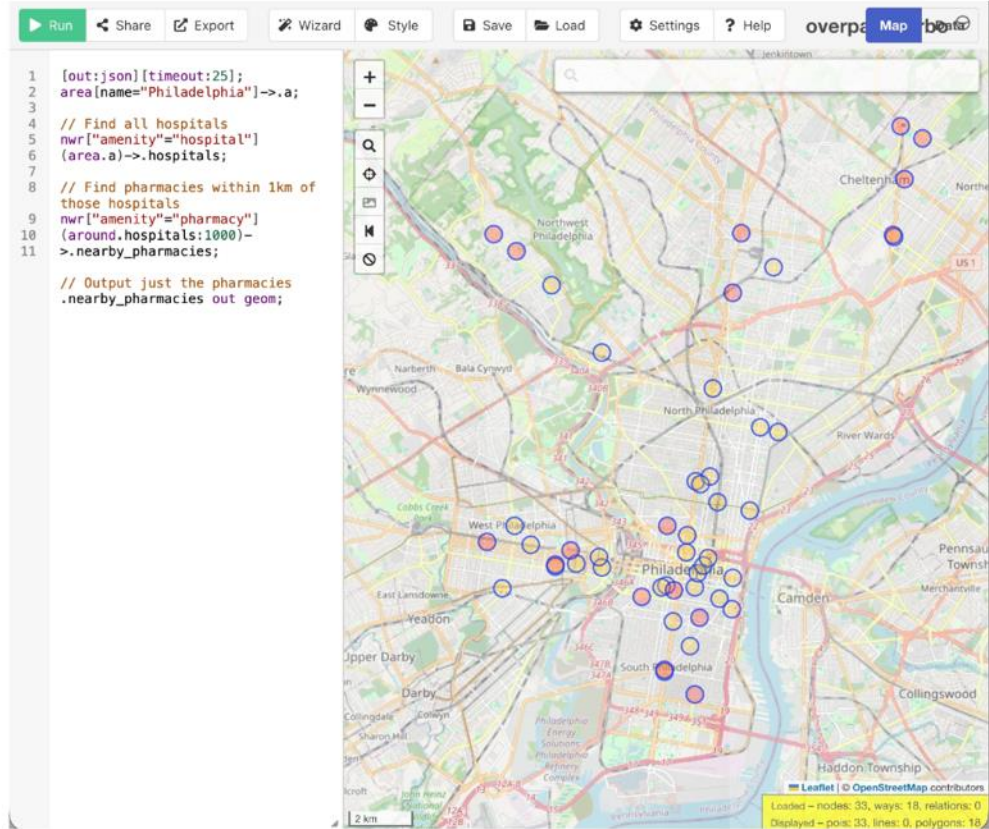
Find pharmacies within 1km from hospitals

```
[out:json][timeout:25];
area[name="Philadelphia"]->.a;

// Find all hospitals
nwr["amenity"="hospital"](area.a)->.hospitals;

// Find pharmacies within 1km of those hospitals
nwr["amenity"="pharmacy"](around.hospitals:1000)->.nearby_pharmacies;

// Output just the pharmacies
.nearby_pharmacies out center;
```



Lets try something out

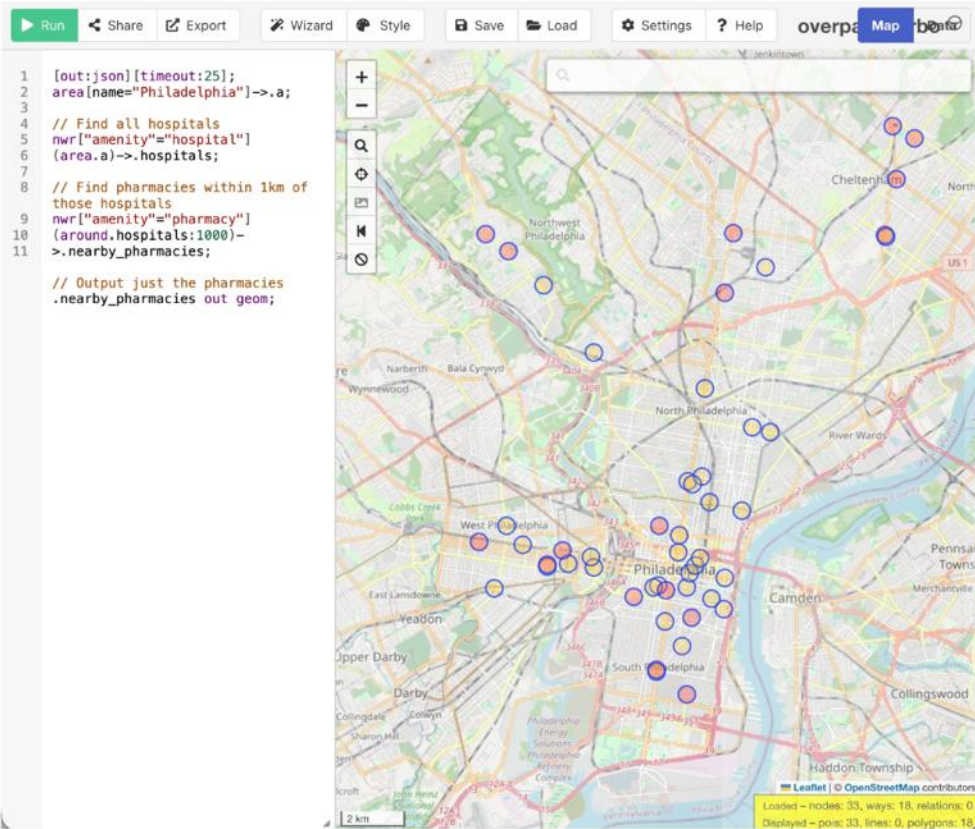
Find pharmacies within 1km from hospitals

```
[out:json][timeout:25];
area[name="Philadelphia"]->.a;

// Find all hospitals
nwr["amenity"="hospital"](area.a)->.hospitals;

// Find pharmacies within 1km of those hospitals
nwr["amenity"="pharmacy"](around.hospitals:1000)->.nearby_pharmacies;

// Output just the pharmacies
.nearby_pharmacies out center;
```



Lets try something out

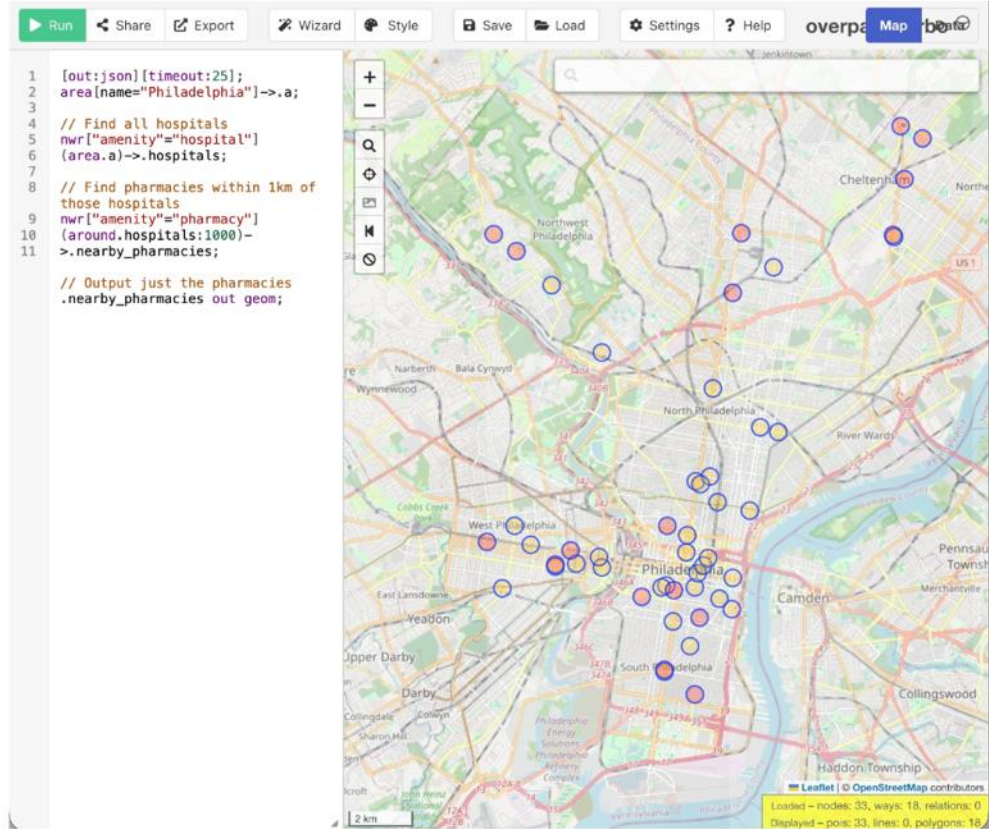
Find pharmacies within 1km from hospitals

```
[out:json][timeout:25];
area[name="Philadelphia"]->.a;

// Find all hospitals
nwr["amenity"="hospital"](area.a)->.hospitals;

// Find pharmacies within 1km of those hospitals
nwr["amenity"="pharmacy"](around.hospitals:1000)->.
nearby_pharmacies;

// Output just the pharmacies
.nearby_pharmacies out center;
```



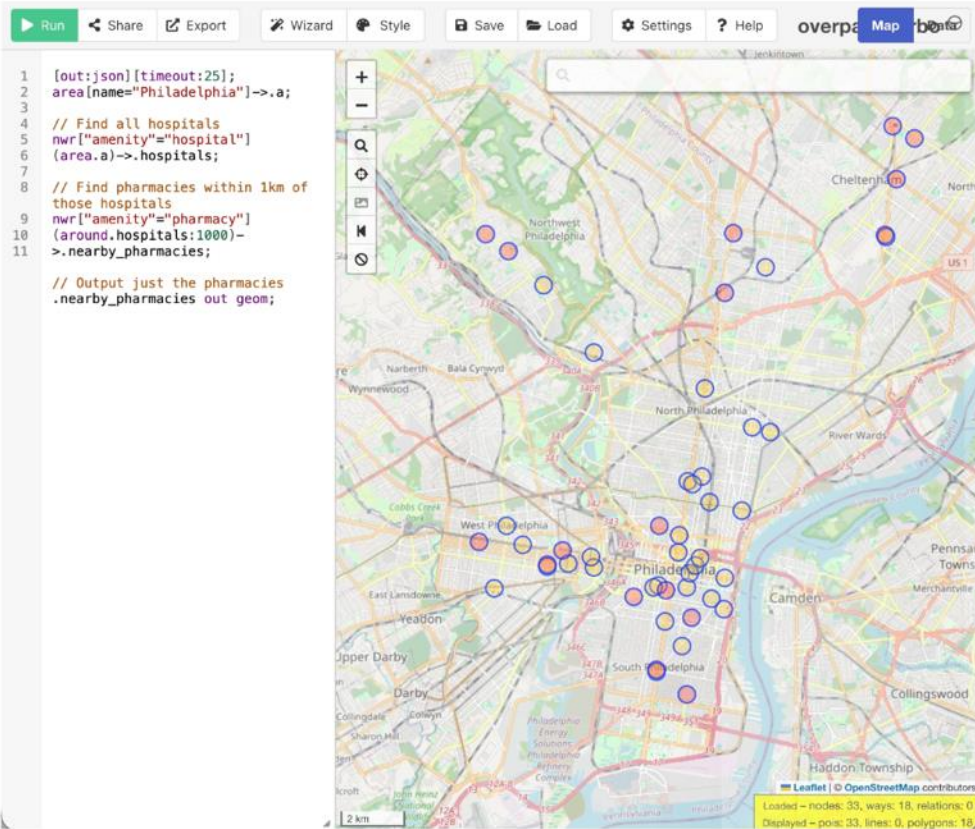
Lets try something out

Find pharmacies within 1km from hospitals

```
[out:json][timeout:25];
area[name="Philadelphia"]->.a;

// Find all hospitals
nwr["amenity"="hospital"](area.a)->.hospitals;

// Find pharmacies within 1km of those hospitals
nwr["amenity"="pharmacy"](around.hospitals:1000)->.nearby_pharmacies out center;
```



Lets try something out

Find pharmacies within 1km from hospitals

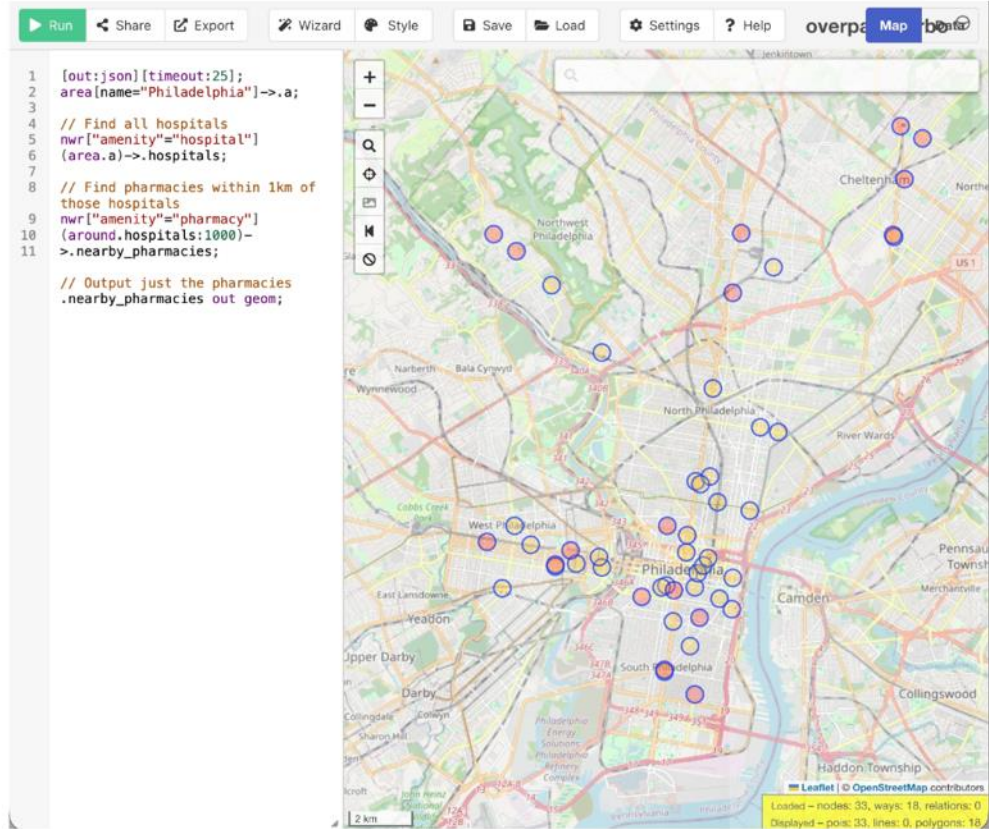
```
[out:json][timeout:25];
area[name="Philadelphia"]->.a;

// Find all hospitals
nwr["amenity"="hospital"](area.a)->.hospitals;

// Find pharmacies within 1km of those hospitals
nwr["amenity"="pharmacy"](around.hospitals:1000)->.nearby_pharmacies;

// Output just the pharmacies
.nearby_pharmacies out center;
```

1. Copy the query



Lets try something out

Find pharmacies within 1km from hospitals

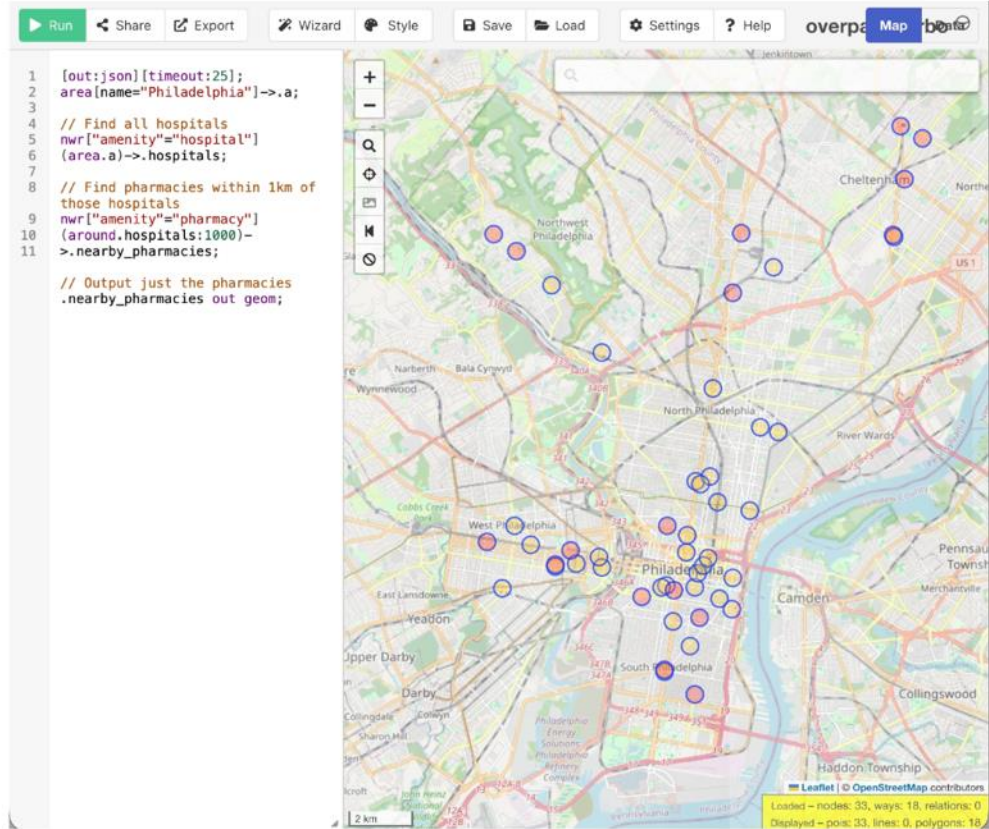
```
[out:json][timeout:25];
area[name="Philadelphia"]->.a;

// Find all hospitals
nwr["amenity"="hospital"](area.a)->.hospitals;

// Find pharmacies within 1km of those hospitals
nwr["amenity"="pharmacy"](around.hospitals:1000)->.nearby_pharmacies;

// Output just the pharmacies
.nearby_pharmacies out center;
```

1. Copy the query
2. Got to <https://overpass-turbo.eu/>



Lets try something out

Find pharmacies within 1km from hospitals

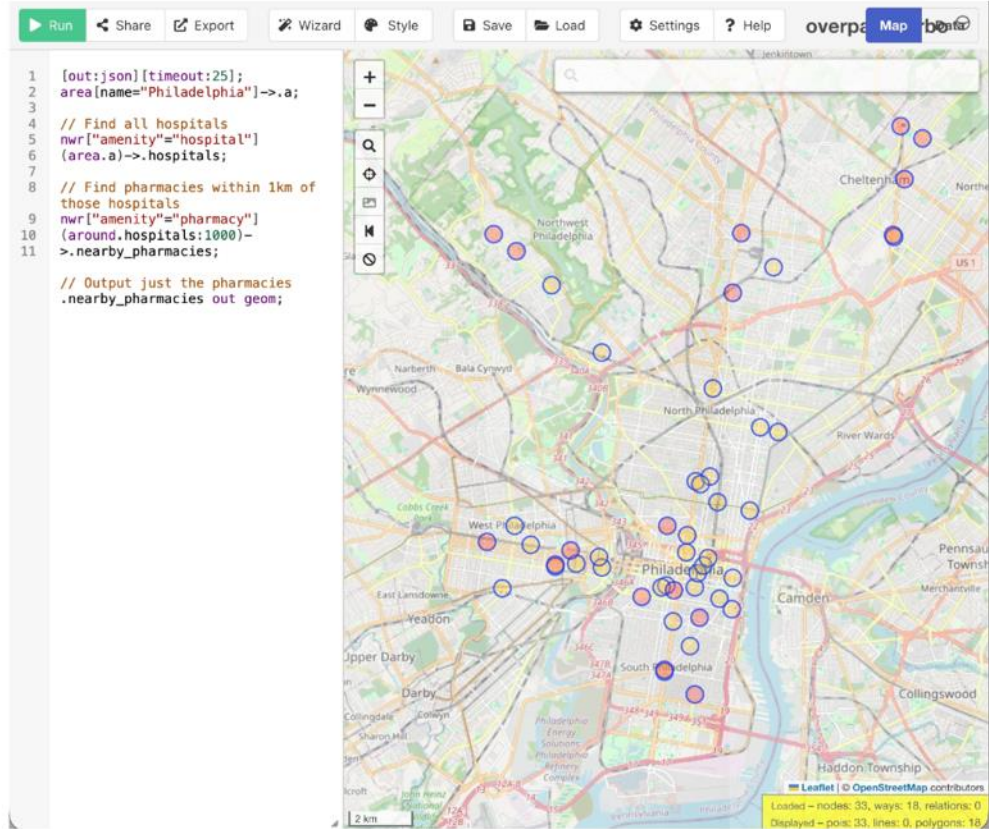
```
[out:json][timeout:25];
area[name="Philadelphia"]->.a;

// Find all hospitals
nwr["amenity"="hospital"](area.a)->.hospitals;

// Find pharmacies within 1km of those hospitals
nwr["amenity"="pharmacy"](around.hospitals:1000)->.nearby_pharmacies;

// Output just the pharmacies
.nearby_pharmacies out center;
```

1. Copy the query
2. Got to <https://overpass-turbo.eu/>
3. Search and zoom in to Philadelphia, PA



Lets try something out

Find pharmacies within 1km from hospitals

```
[out:json][timeout:25];
area[name="Philadelphia"]->.a;

// Find all hospitals
nwr["amenity"="hospital"](area.a)->.hospitals;

// Find pharmacies within 1km of those hospitals
nwr["amenity"="pharmacy"](around.hospitals:1000)->.
nearby_pharmacies out center;
```

1. Copy the query
2. Got to <https://overpass-turbo.eu/>
3. Search and zoom in to Philadelphia, PA
4. Paste the query on the text area on the left

The screenshot shows the Overpass Turbo interface. On the left, a text editor contains the following query:

```
1 [out:json][timeout:25];
2 area[name="Philadelphia"]->.a;
3
4 // Find all hospitals
5 nwr["amenity"="hospital"]
6 (area.a)->.hospitals;
7
8 // Find pharmacies within 1km of
9 those hospitals
10 nwr["amenity"="pharmacy"]
11 (around.hospitals:1000)-
12 >.nearby_pharmacies;
13
14 // Output just the pharmacies
15 .nearby_pharmacies out geom;
```

On the right, a map of Philadelphia is displayed with numerous blue circular markers indicating the locations of pharmacies found within 1km of hospitals. The map includes labels for various neighborhoods like Cheltenham, North Philadelphia, River Wards, and Camden. A scale bar at the bottom left indicates 2 km. At the bottom right, a status bar shows: "Loaded - nodes: 33, ways: 18, relations: 0" and "Displayed - pois: 33, lines: 0, polygons: 18".

Lets try something out

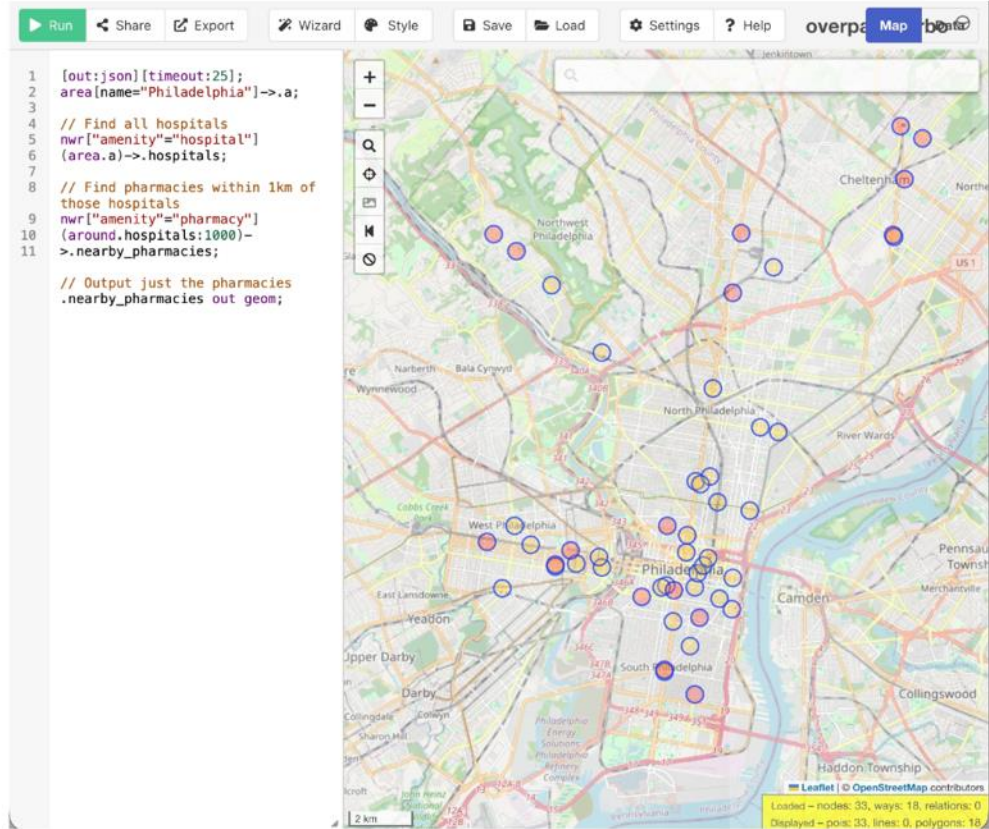
Find pharmacies within 1km from hospitals

```
[out:json][timeout:25];
area[name="Philadelphia"]->.a;

// Find all hospitals
nwr["amenity"="hospital"](area.a)->.hospitals;

// Find pharmacies within 1km of those hospitals
nwr["amenity"="pharmacy"](around.hospitals:1000)->.nearby_pharmacies out center;
```

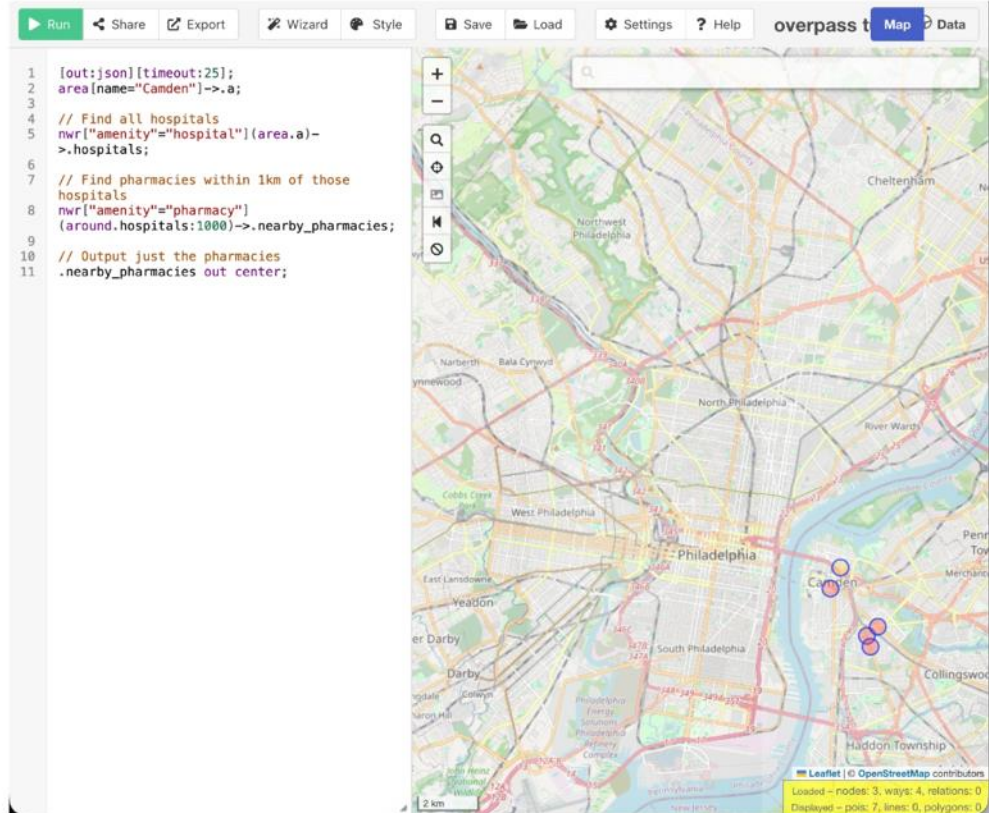
1. Copy the query
2. Got to <https://overpass-turbo.eu/>
3. Search and zoom in to Philadelphia, PA
4. Paste the query on the text area on the left
5. Click the run button



Lets try something out

Change the area name

```
[out:json][timeout:25];  
area[name="Camden"]->.a;  
  
// Find all hospitals  
nwr["amenity"="hospital"](area.a)->.hospitals;  
  
// Find pharmacies within 1km of those hospitals  
nwr["amenity"="pharmacy"](around.hospitals:1000)->.nearby_pharmacies out center;
```



Once again

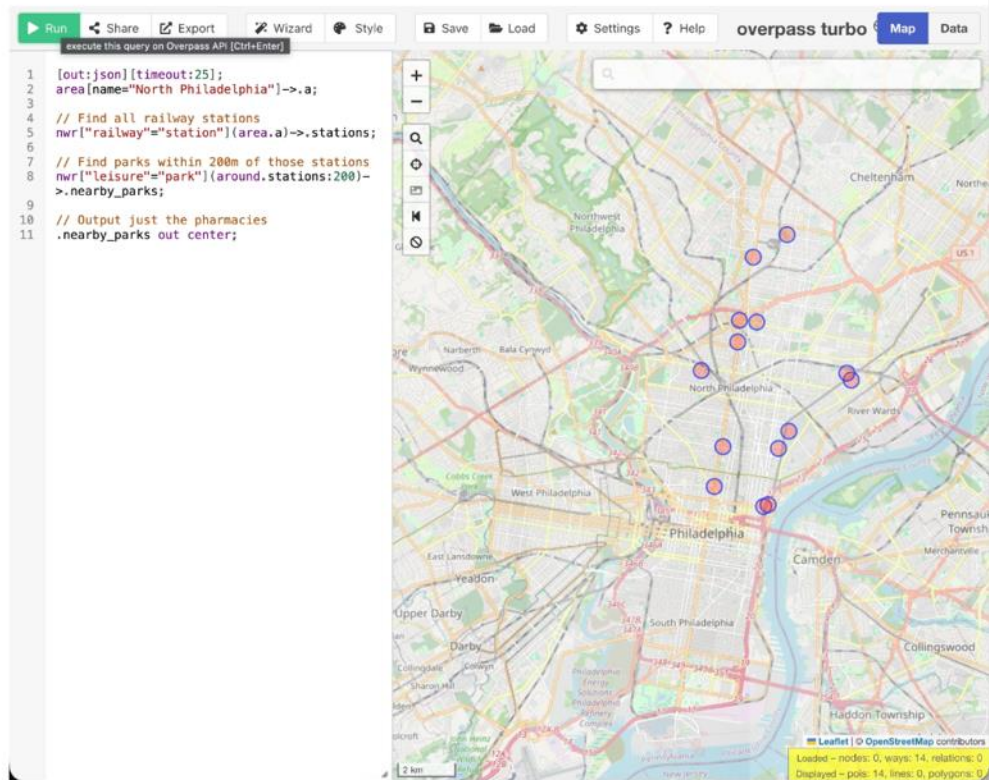
New area, new elements, new distance threshold

```
[out:json][timeout:25];
area[name="North Philadelphia"]->.a;

// Find all railway stations
nwr["railway"="station"](area.a)->.stations;

// Find parks within 200m of those stations
nwr["leisure"="park"](around.stations:200)->.nearby_parks;

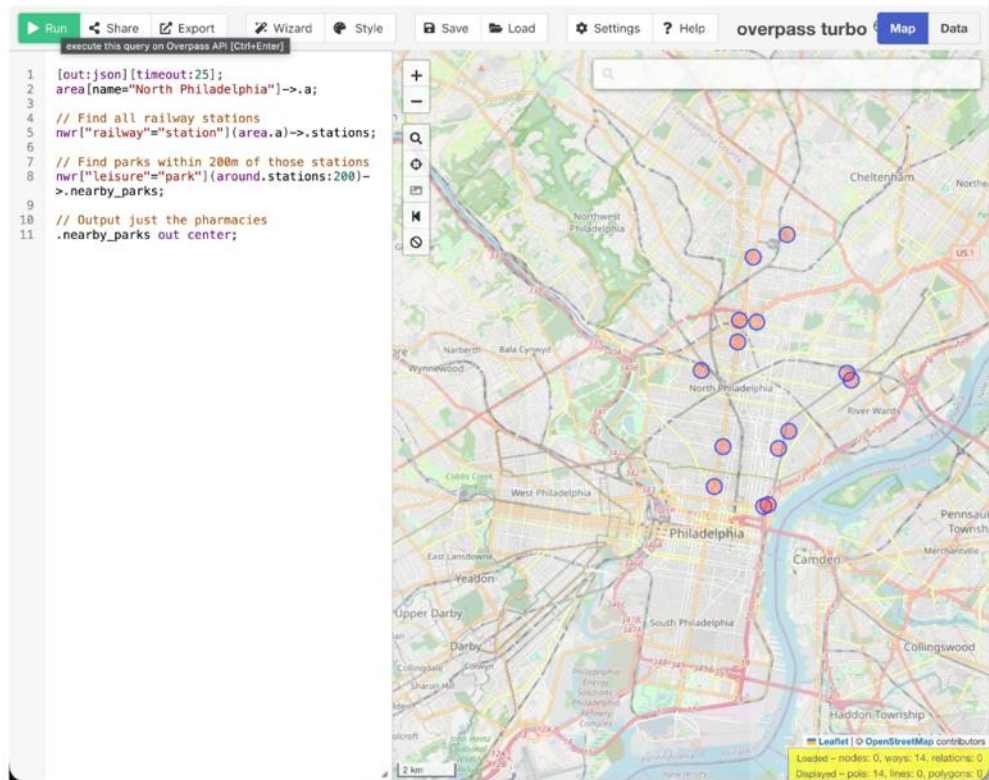
// Output just the pharmacies
.nearby_parks out center;
```



Once again

New area, new elements, new distance threshold

```
[out:json][timeout:25];  
area[name="North Philadelphia"]->.a;  
  
// Find all railway stations  
nwr["railway"="station"](area.a)->.stations;  
  
// Find parks within 200m of those stations  
nwr["leisure"="park"](around.stations:200)->.nearby_parks;  
  
// Output just the pharmacies  
.nearby_parks out center;
```



Once again

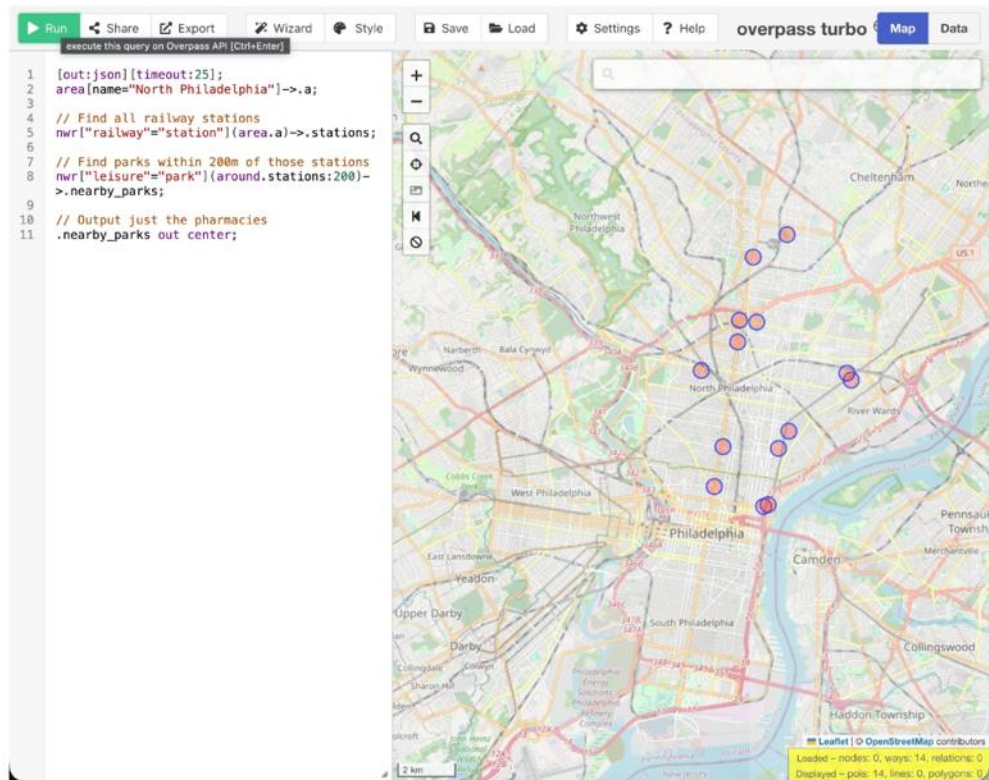
New area, new elements, new distance threshold

```
[out:json][timeout:25];
area[name="North Philadelphia"]->.a;

// Find all railway stations
nwr["railway"="station"](area.a)->.stations;

// Find parks within 200m of those stations
nwr["leisure"="park"](around.stations:200)->.nearby_parks;

// Output just the pharmacies
.nearby_parks out center;
```



Once again

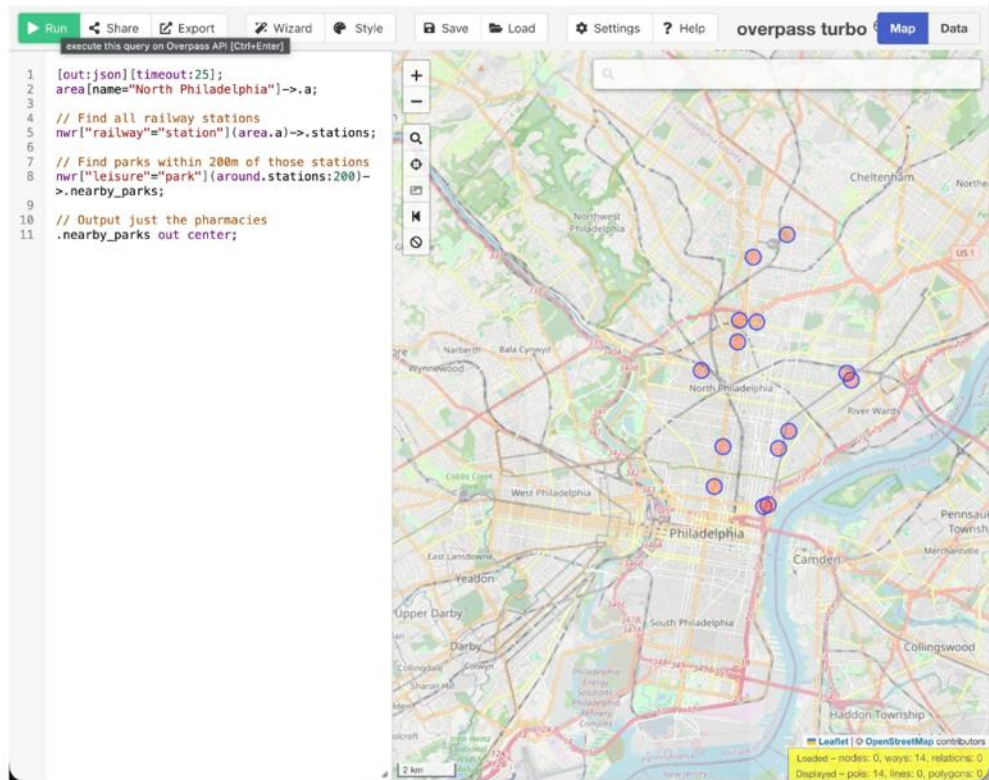
New area, new elements, new distance threshold

```
[out:json][timeout:25];
area[name="North Philadelphia"]->.a;

// Find all railway stations
nwr["railway"="station"](area.a)->.stations;

// Find parks within 200m of those stations
nwr["leisure"="park"](around.stations:200)->.nearby_parks;

// Output just the pharmacies
.nearby_parks out center;
```



Once again

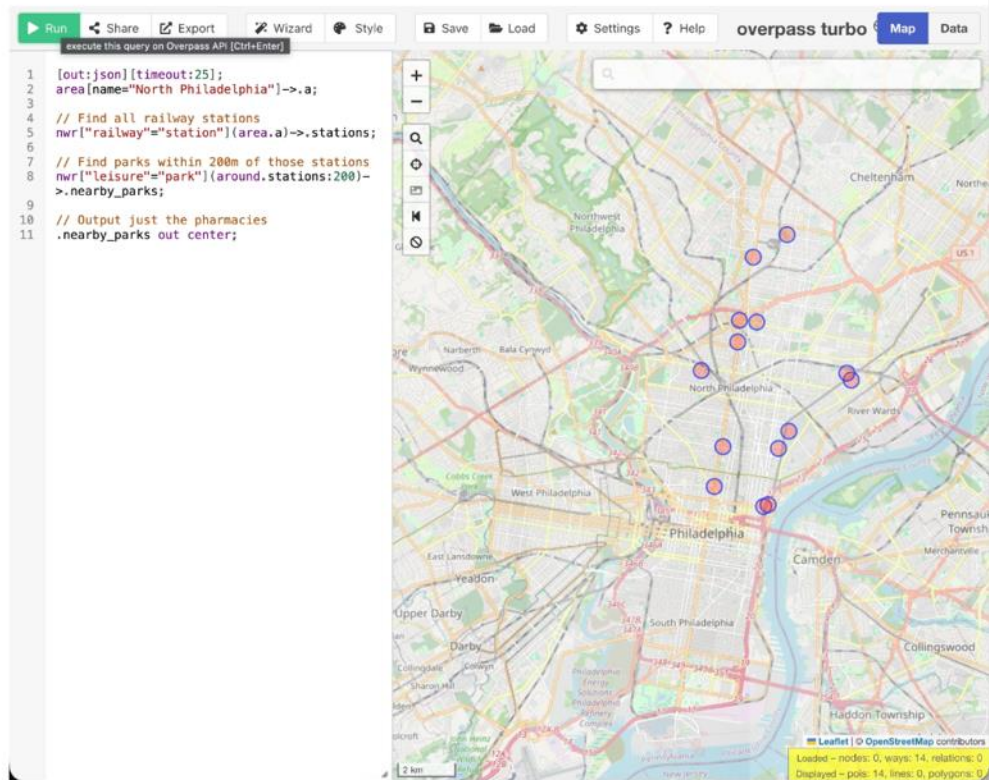
New area, new elements, new distance threshold

```
[out:json][timeout:25];
area[name="North Philadelphia"]->.a;

// Find all railway stations
nwr["railway"="station"](area.a)->.stations;

// Find parks within 200m of those stations
nwr["leisure"="park"](around.stations:200)->.nearby_parks;

// Output just the pharmacies
.nearby_parks out center;
```



Share and Export

You can share and export the results in multiple ways:

1. Share as a link
2. Export the data as: `geojson` `GPX` `KML`
`raw`
3. Export as an image
4. Download/copy the query

 Export 

Data

GeoJSON	download copy
GPX	download copy
KML	download copy
raw OSM data	download copy

raw data directly from [Overpass API](#) 

load data into an OSM editor: [JOSM](#), [Level0](#) 

Map

as [png image](#)

as [interactive Map](#)

current [map view](#) (bbox, center, etc.)

Query

<code>standalone.query</code>	download copy
<code>raw.query</code>	download copy
<code>osm.wiki</code>	download copy

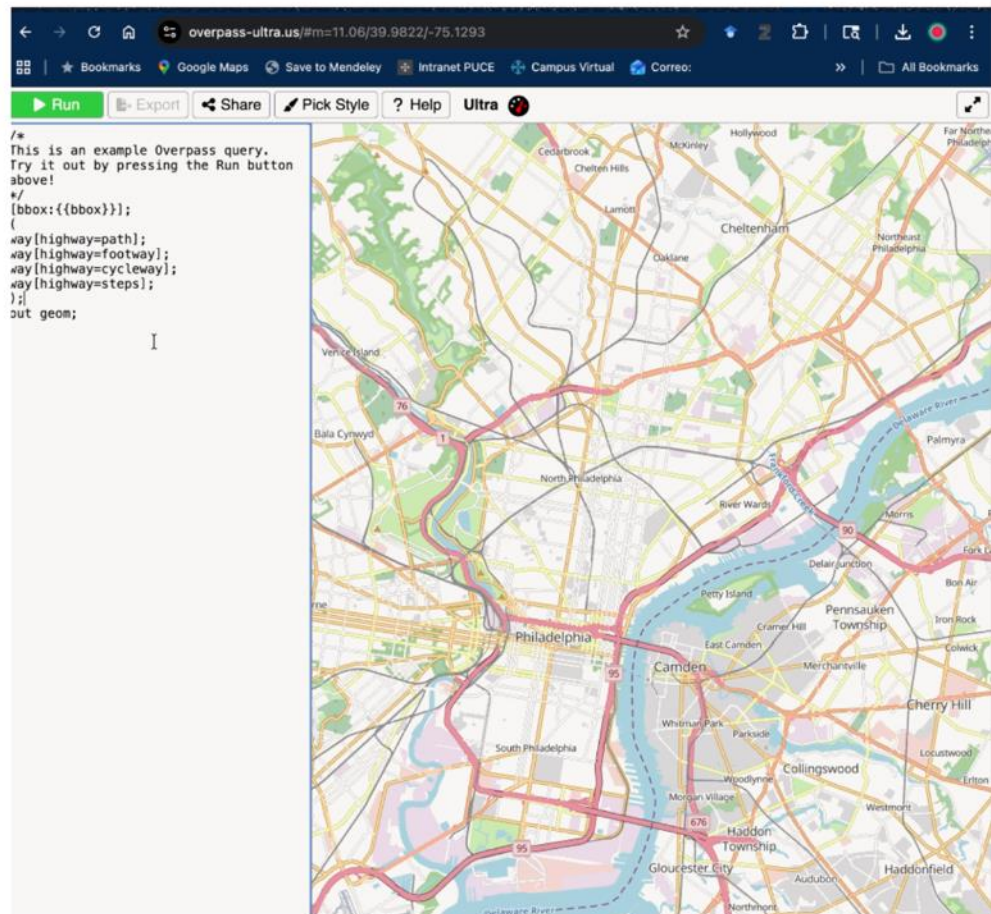
cancel

More styling options:

Use Overpass Ultra

Overpass Ultra query

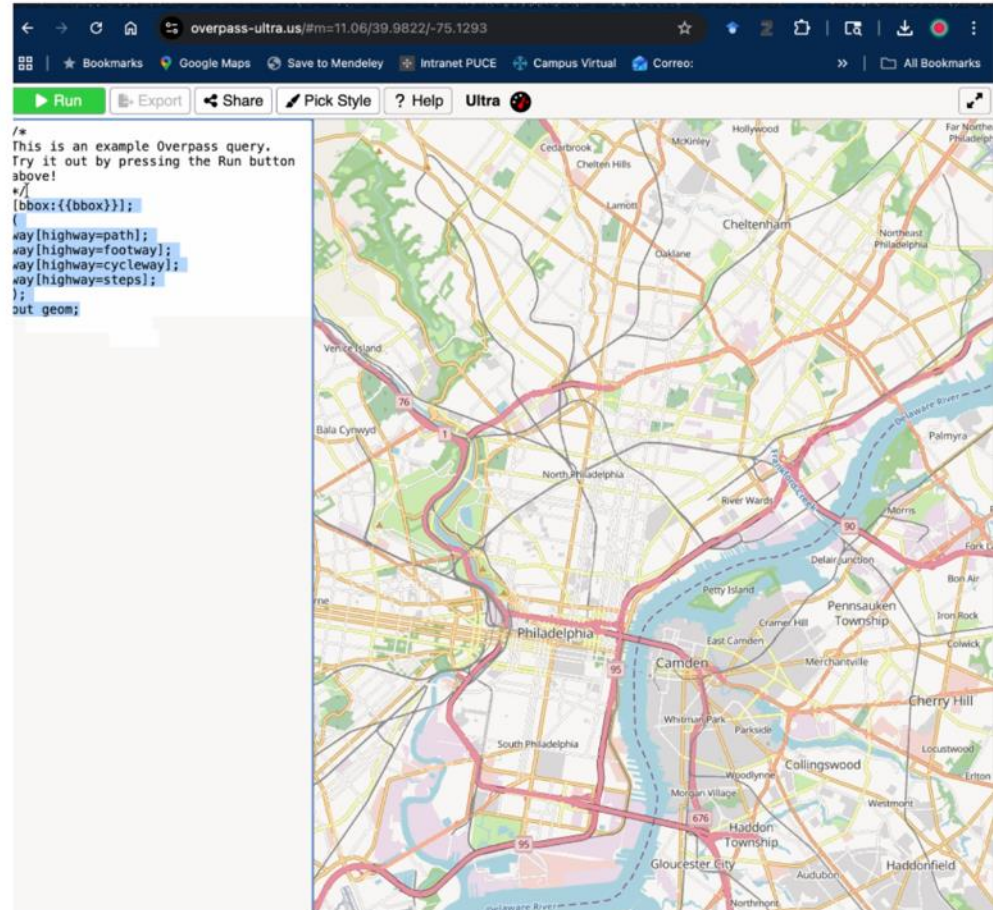
```
---
server: https://overpass-api.de/api/interpreter
style:
  layers:
    - type: heatmap
      paint:
        heatmap-opacity: 0.5
        heatmap-intensity: 1
        heatmap-radius: 5
  extends: https://api.protomaps.com/styles/v5/gray:
options:
  zoom: 11
  center: [-75.1293, 39.9822]
---
[bbox:{{bbox}}];
nwr['leisure'="swimming_pool"];
out center;
```



Overpass Ultra query

```
---
server: https://overpass-api.de/api/interpreter
style:
  layers:
    - type: heatmap
      paint:
        heatmap-opacity: 0.5
        heatmap-intensity: 1
        heatmap-radius: 5
  extends: https://api.protomaps.com/styles/v5/gray
options:
  zoom: 11
  center: [-75.1293, 39.9822]
---
[bbox:{{bbox}}];
nwr['leisure'="swimming_pool"];
out center;
```

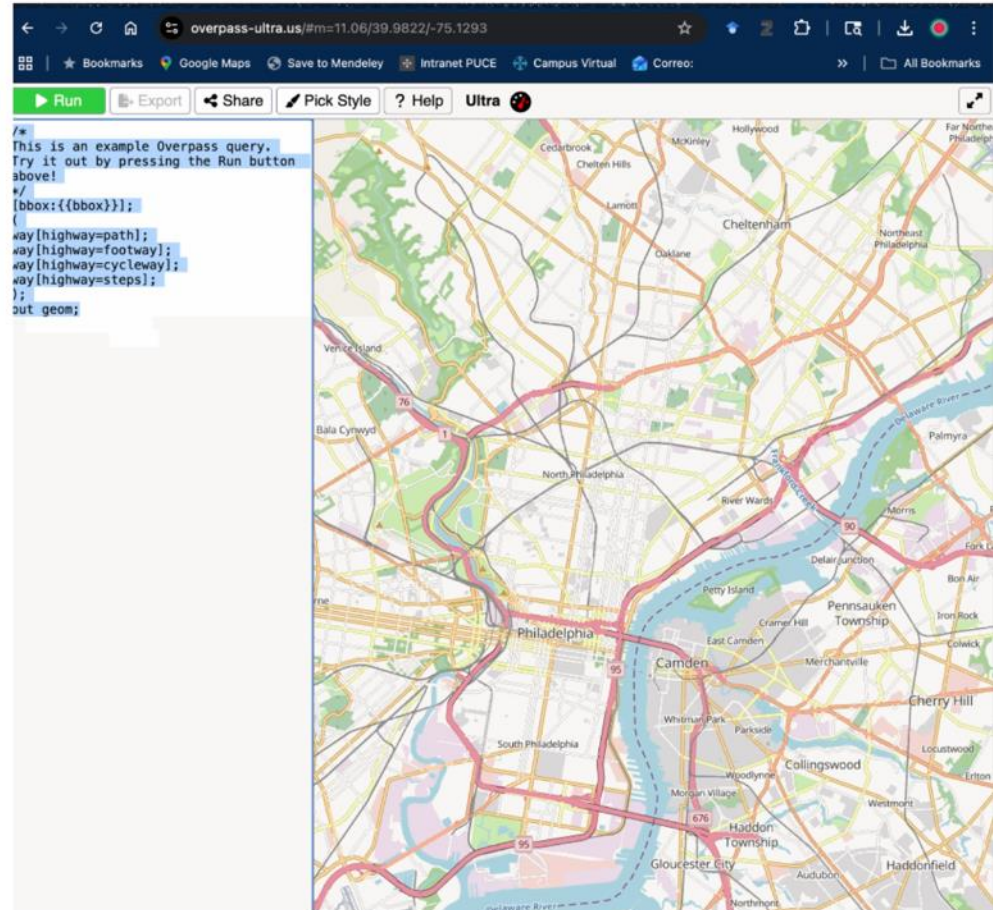
1. Go to <https://overpass-ultra.us/>



Overpass Ultra query

```
---
server: https://overpass-api.de/api/interpreter
style:
  layers:
    - type: heatmap
      paint:
        heatmap-opacity: 0.5
        heatmap-intensity: 1
        heatmap-radius: 5
  extends: https://api.protomaps.com/styles/v5/gray:
options:
  zoom: 11
  center: [-75.1293, 39.9822]
---
[ bbox: {{bbox}} ];
nwr['leisure'="swimming_pool"];
out center;
```

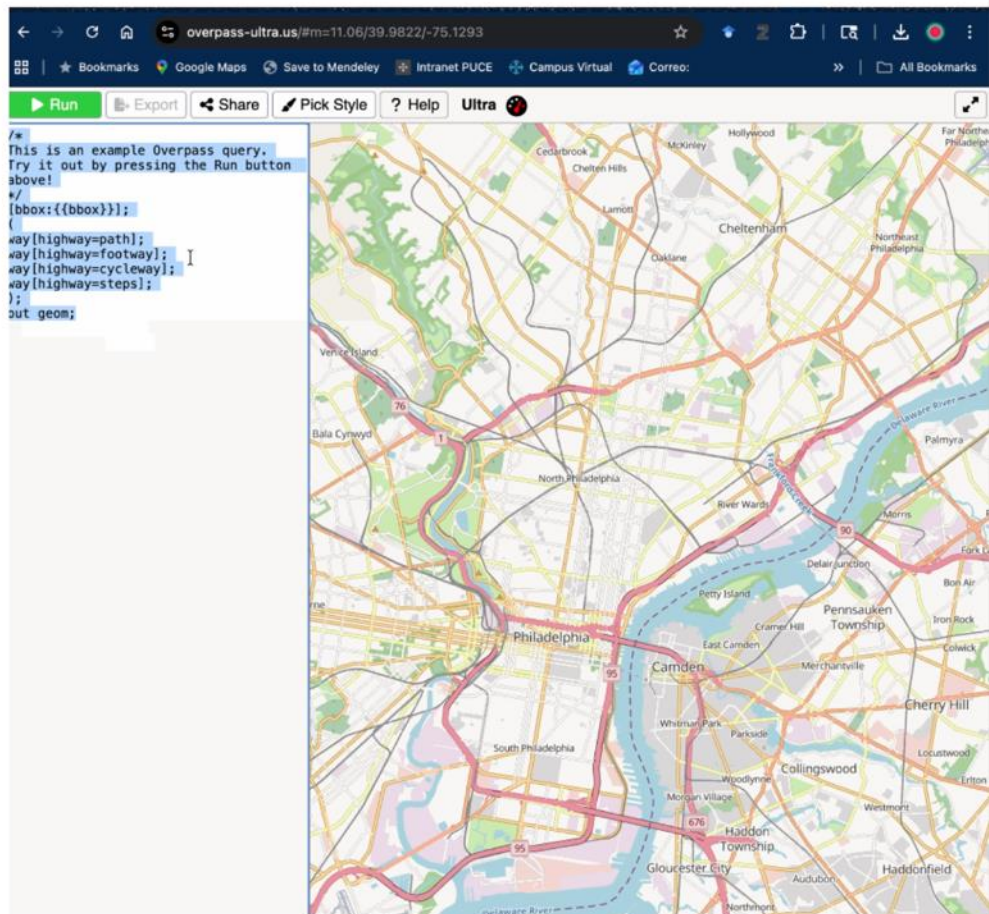
1. Go to <https://overpass-ultra.us/>
2. Create a query (or paste one you already created in turbo)



Overpass Ultra query

```
---
server: https://overpass-api.de/api/interpreter
style:
  layers:
    - type: heatmap
      paint:
        heatmap-opacity: 0.5
        heatmap-intensity: 1
        heatmap-radius: 5
  extends: https://api.protomaps.com/styles/v5/gray:
options:
  zoom: 11
  center: [-75.1293, 39.9822]
---
[ bbox: {{bbox}} ];
nwr['leisure'="swimming_pool"];
out center;
```

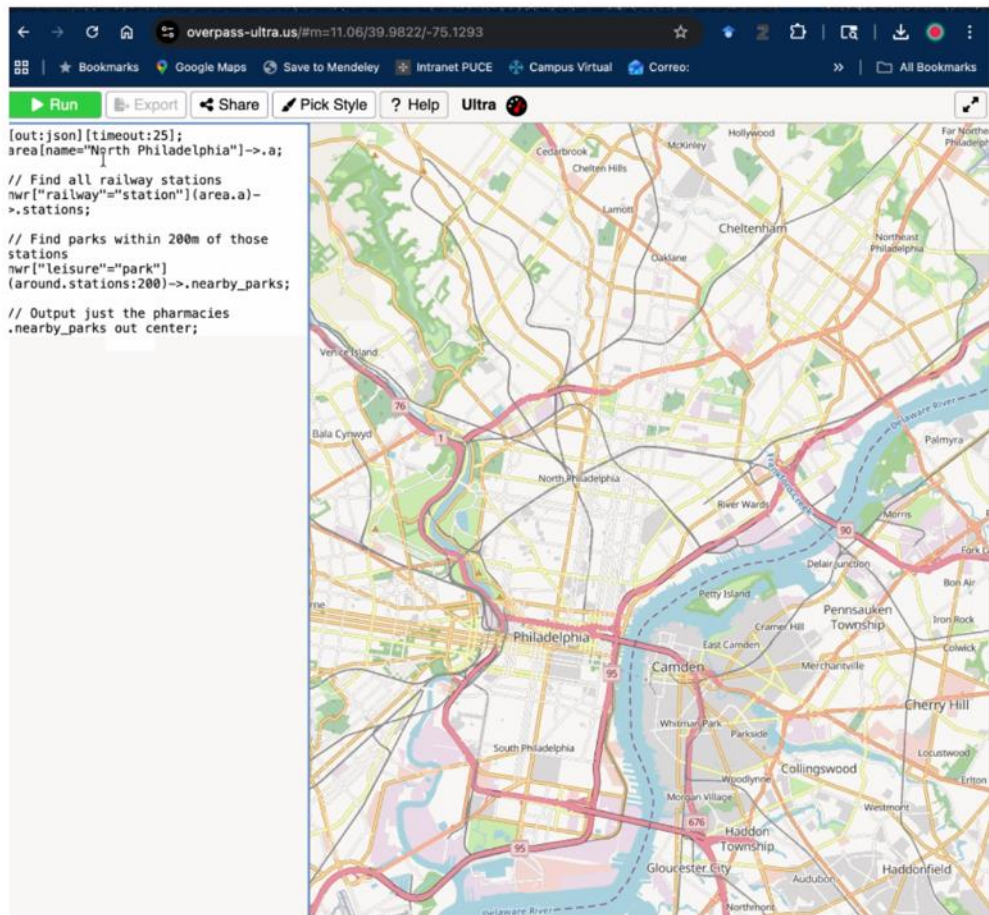
1. Go to <https://overpass-ultra.us/>
2. Create a query (or paste one you already created in turbo)
3. Pick a different basemap style

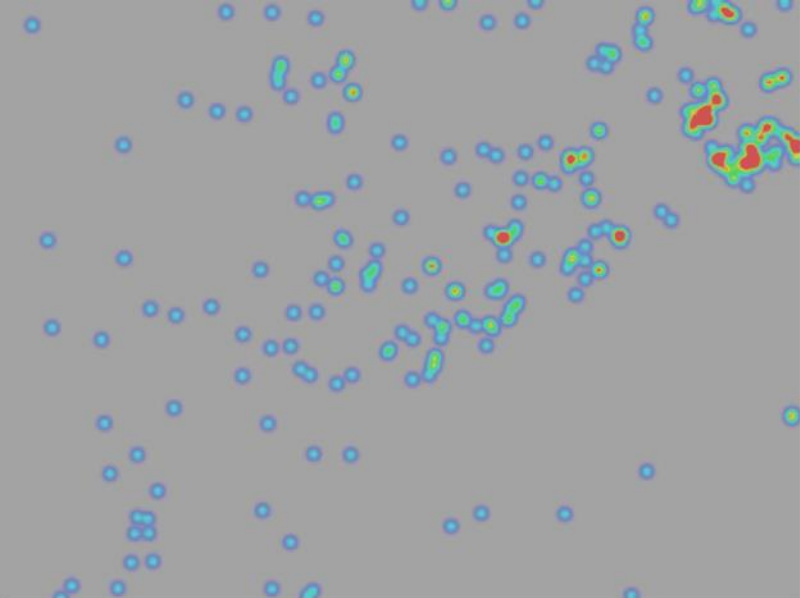


Overpass Ultra query

```
---
server: https://overpass-api.de/api/interpreter
style:
  layers:
    - type: heatmap
      paint:
        heatmap-opacity: 0.5
        heatmap-intensity: 1
        heatmap-radius: 5
  extends: https://api.protomaps.com/styles/v5/gray
options:
  zoom: 11
  center: [-75.1293, 39.9822]
---
[ bbox: {{bbox}} ];
nwr[ 'leisure' = "swimming_pool" ];
out center;
```

1. Go to <https://overpass-ultra.us/>
2. Create a query (or paste one you already created in turbo)
3. Pick a different basemap style
4. Add more styling using the YAML frontmatter





Styling on Ultra

Step-by-step

```
1 //Step 1: add an Overpass Query
2 [bbox:{{bbox}}];
3 nwr['leisure'="swimming_pool"];
4 out center;
```

Styling on Ultra

Step-by-step

```
1 //Step 2: add a YAML front matter
2 ---
3
4 ---
5 [bbox:{{bbox}}];
6 nwr['leisure']="swimming_pool";
7 out center;
```


Styling on Ultra

Step-by-step

```
1 //Step 4: add a center and a zoom
2 ---
3 style:
4   layers:
5     - type: heatmap
6       paint:
7         heatmap-opacity: 0.5
8         heatmap-intensity: 1
9         heatmap-radius: 5
10    extends: https://api.protomaps.com/styles/v5/grayscale/en.json?key=f8eceeaaad6896c92
11  options:
12    zoom: 11
13    center: [-75.1293, 39.9822]
14  ---
15  [bbox:{{bbox}}];
16  nwr['leisure'='swimming_pool'];
17  out center;
```

Styling on Ultra

Step-by-step

```
1 //Step 4: add a center and a zoom
2 ---
3 style:
4   layers:
5     - type: heatmap
6       paint:
7         heatmap-opacity: 0.5
8         heatmap-intensity: 1
9         heatmap-radius: 5
10    extends: https://api.protomaps.com/styles/v5/grayscale/en.json?key=f8eceeaaad6896c92
11  options:
12    zoom: 11
13    center: [-75.1293, 39.9822]
14  ---
15  [bbox:{{bbox}}];
16  nwr['leisure'='swimming_pool'];
17  out center;
```

Styling on Ultra

Step-by-step

```
1 //Step 4: add a center and a zoom
2 ---
3 style:
4   layers:
5     - type: heatmap
6       paint:
7         heatmap-opacity: 0.5
8         heatmap-intensity: 1
9         heatmap-radius: 5
10    extends: https://api.protomaps.com/styles/v5/grayscale/en.json?key=f8eceeaaad6896c92
11  options:
12    zoom: 11
13    center: [-75.1293, 39.9822]
14  ---
15  [bbox:{{bbox}}];
16  nwr['leisure'='swimming_pool'];
17  out center;
```

Styling on Ultra

Step-by-step

```
1 //Step 4: add a center and a zoom
2 ---
3 style:
4   layers:
5     - type: heatmap
6       paint:
7         heatmap-opacity: 0.5
8         heatmap-intensity: 1
9         heatmap-radius: 5
10    extends: https://api.protomaps.com/styles/v5/grayscale/en.json?key=f8eceeaaad6896c92
11  options:
12    zoom: 11
13    center: [-75.1293, 39.9822]
14  ---
15  [bbox:{{bbox}}];
16  nwr['leisure'="swimming_pool"];
17  out center;
```

MAPPING SERIES WORKSHOPS



Introduction to QGIS: Open Source mapping software
Wednesday, January 21, 3pm



Web Mapping for Non-programmers
Wednesday, February 4, 3pm



Mapping Philadelphia in OpenStreetMap
Wednesday, February 11, 12pm



Using OpenStreetMap for Urban Spatial Analysis
Wednesday, February 18, 3pm



Field Data Collection & Mobile GIS
Wednesday, March 11, 3pm



From Historical Maps to Digital Stories
Wednesday, March 25, 3pm



Mapping and Geospatial Data Visualization with Python
Wednesday, April 15, 3pm

Learn more and register at: library.temple.edu/events

Contact us at:
felipe.valdez@temple.edu

Visit our guides:
<https://guides.temple.edu/gis-mapping>